Homework6 ARIMA

Mehreen Ali Gillani

2026-03-16

Exercises 9.1, 9.2, 9.3, 9.5, 9.6, 9.7, 9.8 in Hyndman

9.1 Figure 9.32 shows the ACFs for 36 random numbers, 360 random numbers and 1,000 random numbers. a. Explain the differences among these figures. Do they all indicate that the data are white noise?

Answer:

Yes, all three series are white noise. But they look different because the confidence band width depends on sample size n, and with smallsamples, random fluctuations can accidentally cross the bands even in true white noise.

- Left panel sample size, n = 36: Yes — with only 36 observations, random chance alone will occasionally produce a spike near the boundary. At 5% significance level, you statistically expect 1 false crossing out of every 20 lags tested.
- Middle Panel — n = 360 (Medium Sample): Is it white noise? Yes — all autocorrelations are statistically insignificant
- Right panel n = 1000: Yes — with 1,000 observations, the estimator is very precise, confirming there is truly no autocorrelation structure.

Key Insight — Why Small Samples Look "Less White Noise"

This is a sampling variability phenomenon. Even if the true population autocorrelation at every lag is exactly zero, your sample autocorrelation will never be exactly zero — it will wobble randomly around zero.

$\rho k \approx 0 \pm 1.96 n \hat{\rho}_k \approx 0 \pm \frac{1.96}{\sqrt{n}} \rho k \approx 0 \pm n 1.96$

- With small n, the wobble is large → spikes look big → appears to havestructure but doesn't
- With large n, the wobble is tiny → spikes look small → true white noise nature is clearly visible

b. Why are the critical values at different distances from the mean of zero? Why are the autocorrelations different in each figure when they each refer to white noise? Why Are Critical Values at Different Distances from Zero? Because the confidence bands formula is: $\pm 1.96 n \pm \frac{1.96}{\sqrt{n}} \pm n 1.96$ As n increases, the bands get narrower — mathematically:

panel	n	band width
Left	36	±1.96√ 36=±0.327
Middle	360	±1.96√ 360 = ±0.103
Right	1000	±1.96√ 1000 = ±0.062

More data = more certainty = tighter bands. With larger samples you can detect even tiny real autocorrelations, so the threshold for significance shrinks.

Why Are Autocorrelations Different if All Three Are White Noise?

Because of sampling variability. Even when the true population autocorrelation is exactly zero at every lag, the sample autocorrelation will never be exactly zero — it randomly wobbles around zero:

$\rho k \approx 0 \pm 1 n \hat{\rho}_k \approx 0 \pm \frac{1}{\sqrt{n}} \rho k \approx 0 \pm n 1$

Panel	n	Wobble Size	What You See
Left	36	Large 1√ 36=0.167	Spikes look tall and irregular
Middle	360	Medium 1√360=0.053	Spikes are moderate
Right	1,000	Small 1√ 1000=0.032	Spikes are tiny, almost flat

Both questions have same root cause

- Small n → large sampling variability → spikes look big → wide confidence bands
- Large n → small sampling variability → spikes look small → narrow confidence bands

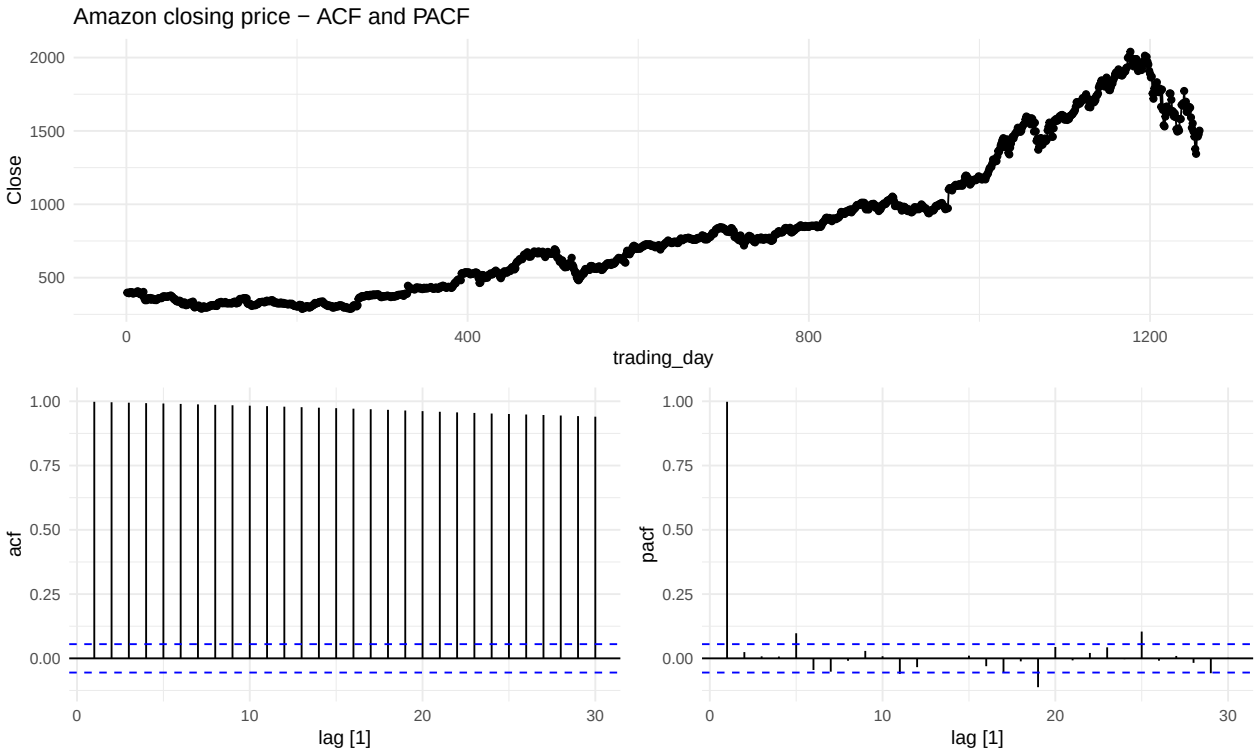
The bands and the spike heights both shrink as n grows because *larger samples estimate auto-correlations more precisely*— but all three are still white noise because the spikes are due to *random sampling fluctuation, not real structure.*

```r
# load data and plot closing price
# Filter Amazon stock from gafa_stock dataset
amazon <- gafa_stock |>
  filter(Symbol == "AMZN") |>
  mutate(trading_day = row_number()) |>
  update_tsibble(index = trading_day, regular = TRUE)

# Plot closing price
amazon |> autoplot(Close) +
  labs(title = "Amazon daily closing price",
       y = "Closing price (USD)", x = "Trading day")
```

9.2 A classic example of a non-stationary series are stock prices. Plot the daily closing prices for Amazon stock (contained in gafa_stock), along with the ACF and PACF. Explain how each plot shows that the series is non-stationary and should be differenced.

The time plot show a clear upward trend — the mean is not constant, it drifts upward over time. This alone is enough to conclude the series is non-stationary and needs differencing.

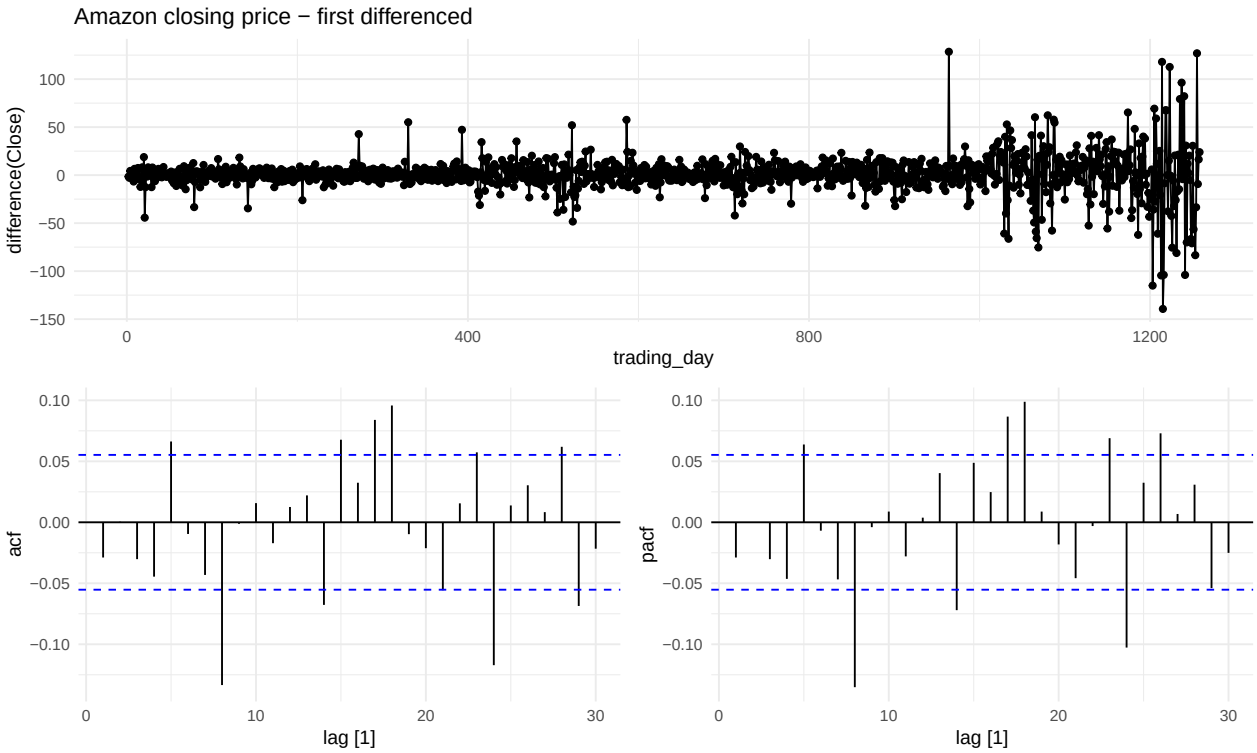
```
# ACF and PACF together using gg_tsdisplay
amazon |>
  ggtime::gg_tsdisplay(Close, plot_type = "partial") +
  labs(title = "Amazon closing price - ACF and PACF")
```



Time plot

- Clear upward trend from ~$300 to over $2000. Mean is increasing, not constant over time. Non-stationary: trending mean
- ACF plot Autocorrelations start near 1.0 at lag 1 and decay very slowly across many lags. Stays above the blue bands for a long time. Non-stationary: slow ACF decay
- PACF plot One very large spike at lag 1 (near 1.0), then all other lags cut off sharply inside the bands. Signature of a random walk / unit root

```r
# First difference and plot
amazon |>
  gg_tsdisplay(difference(Close), plot_type = "partial") +
  labs(title = "Amazon closing price - first differenced")
```



Time Plot (top):

- After differencing, the series fluctuates around zero with no upward or downward trend
- However, the variance is NOT constant — early trading days (left side) have small fluctuations (±10), but later trading days (right side, around day 1000–1250) have very large spikes reaching ±150
- This is called heteroskedasticity — changing variance over time

ACF Plot (bottom left)

- Confidence bands are at approximately ±0.055
- multiple spikes crossing upper and lower band
- Spikes are scattered at many lags with no clean cutoff pattern

PACF Plot (bottom right)

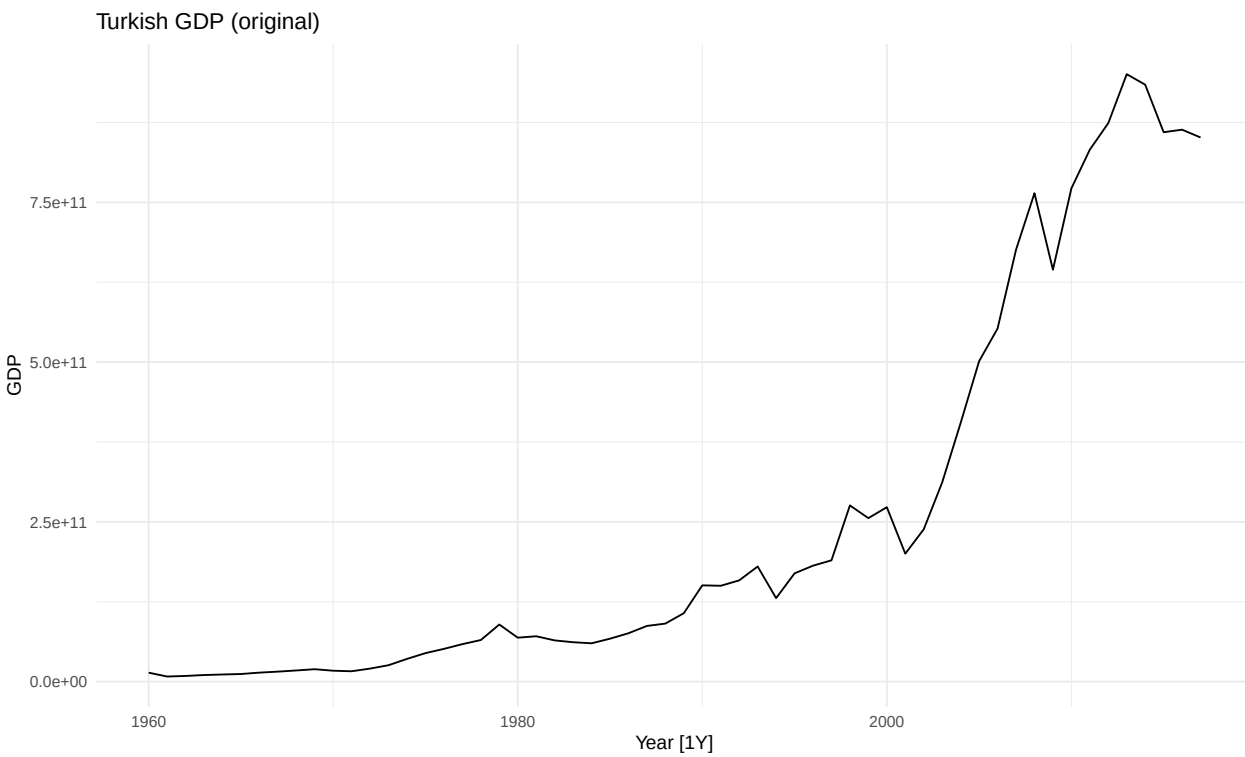
- Very similar pattern to ACF
- multiple spikes crossing upper and lower band
- Again no clean cutoff

After first differencing, the time plot fluctuates around zero confirming the trend has been removed and d = 1 is appropriate. However, the differenced series is not white noise — both the ACF and PACF show multiple significant spikes outside the blue confidence bands. Additionally, the time plot reveals increasing variance in later trading days, indicating heteroskedasticity which ARIMA alone cannot fully capture.

9.3 For the following series, find an appropriate Box-Cox transformation and order of differencing in order to obtain stationary data. Turkish GDP from global_economy. Accommodation takings in the state of Tasmania from aus_accommodation. Monthly sales from souvenirs.

a. Series 1 — Turkish GDP from global_economy

```r
# Filter Turkish GDP
turkey <- global_economy |>
  filter(Country == "Turkey")

turkey |> autoplot(GDP) +
  labs(title = "Turkish GDP (original)")
```

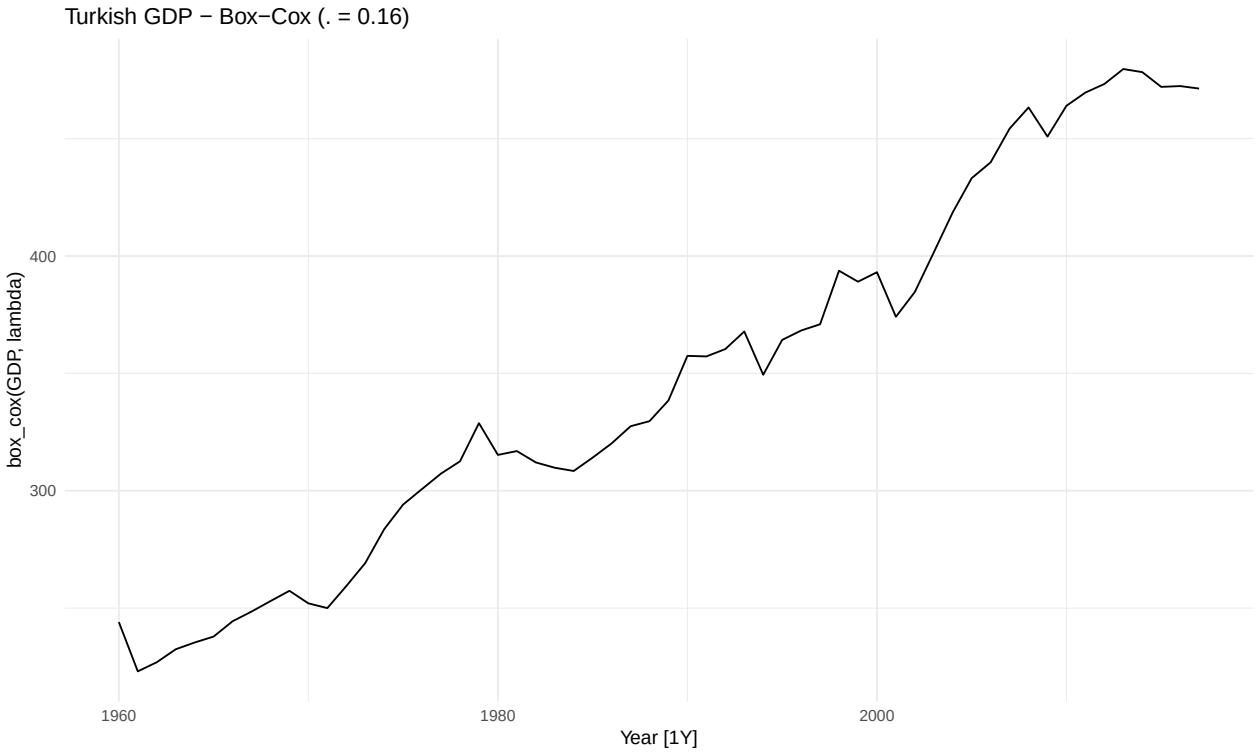


The original series shows a strong upward trend AND increasing variance — the fluctuations grow larger as GDP rises. Both issues need fixing.

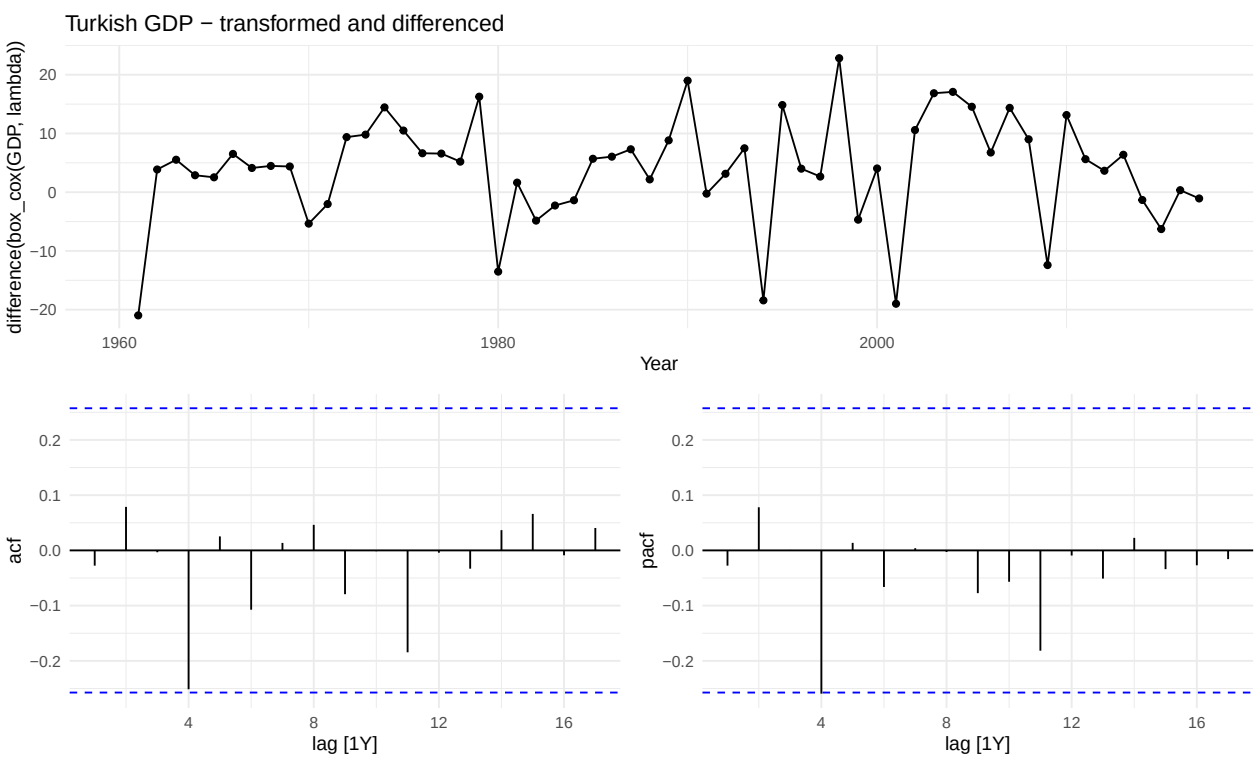
```r
# Find optimal lambda
lambda <- turkey |>
  features(GDP, features = guerrero) |>
  pull(lambda_guerrero)

lambda # check the value - expect close to 0 (log)
```

```
## [1] 0.1572187
```

```r
# Plot Box-Cox transformed series
turkey |> autoplot(box_cox(GDP, lambda)) +
  labs(title = paste0("Turkish GDP - Box-Cox (  = ", round(lambda,2), ")"))
```




```r
# First difference of Box-Cox transformed series
turkey |>
  gg_tsdisplay(difference(box_cox(GDP, lambda)),
               plot_type = "partial") +
  labs(title = "Turkish GDP - transformed and differenced")
```



```r
# Confirm with KPSS test
turkey |>
  mutate(gdp_bc = box_cox(GDP, lambda)) |>
  features(difference(gdp_bc), unitroot_kpss)
```

```
## # A tibble: 1 x 3
##   Country kpss_stat kpss_pvalue
##   <fct>       <dbl>       <dbl>
## 1 Turkey     0.0889         0.1
```

lambda = 0.157 Slightly stronger than log — closer to a weak power transformation

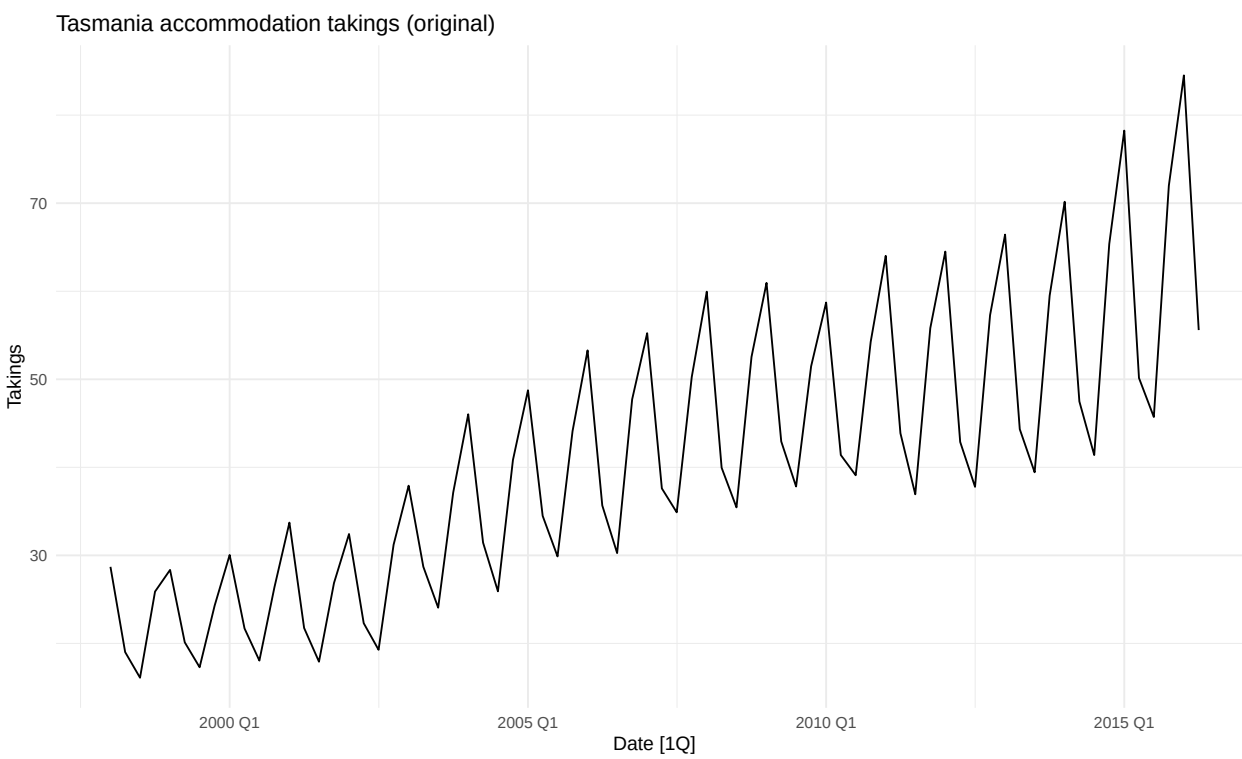
KPSS statistic 0.0889 Very small — close to zero p-value 0.1 Greater than 0.05

p-value = 0.1 > 0.05 → Fail to reject null hypothesis → Series IS stationary

For Turkish GDP, the Guerrero method gives = 0.157, indicating a mild power transformation to stabilise the growing variance. After applying the Box-Cox transformation and first differencing (d = 1), the KPSS test returns a p-value of 0.1 > 0.05, confirming the series is now stationary. Therefore, one order of differencing is sufficient.

b. Series 2 — Tasmania accommodation takings from aus_accommodation

```r
# Filter Tasmania
tasmania <- aus_accommodation |>
  filter(State == "Tasmania")

tasmania |> autoplot(Takings) +
  labs(title = "Tasmania accommodation takings (original)")
```



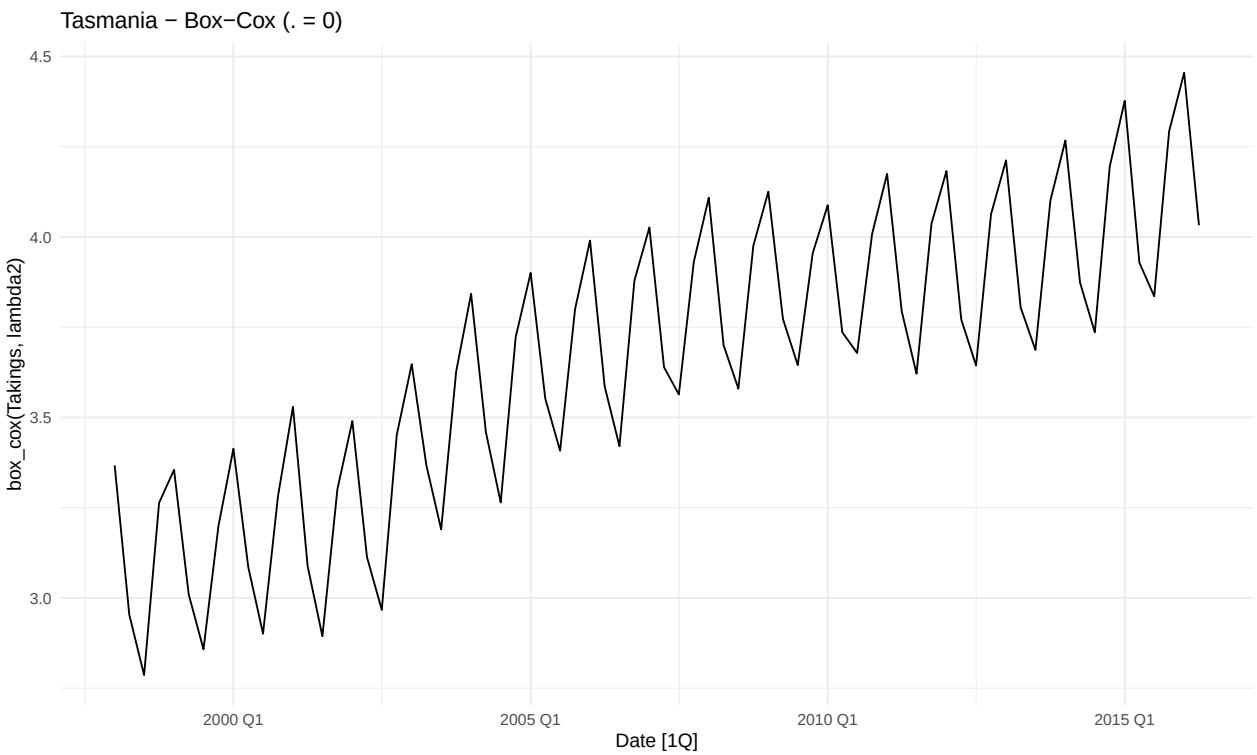
- The series has an upward trend AND strong seasonality (quarterly pattern repeating every year).
- The seasonal swings also grow larger over time — increasing variance.

BOX-COX transformation

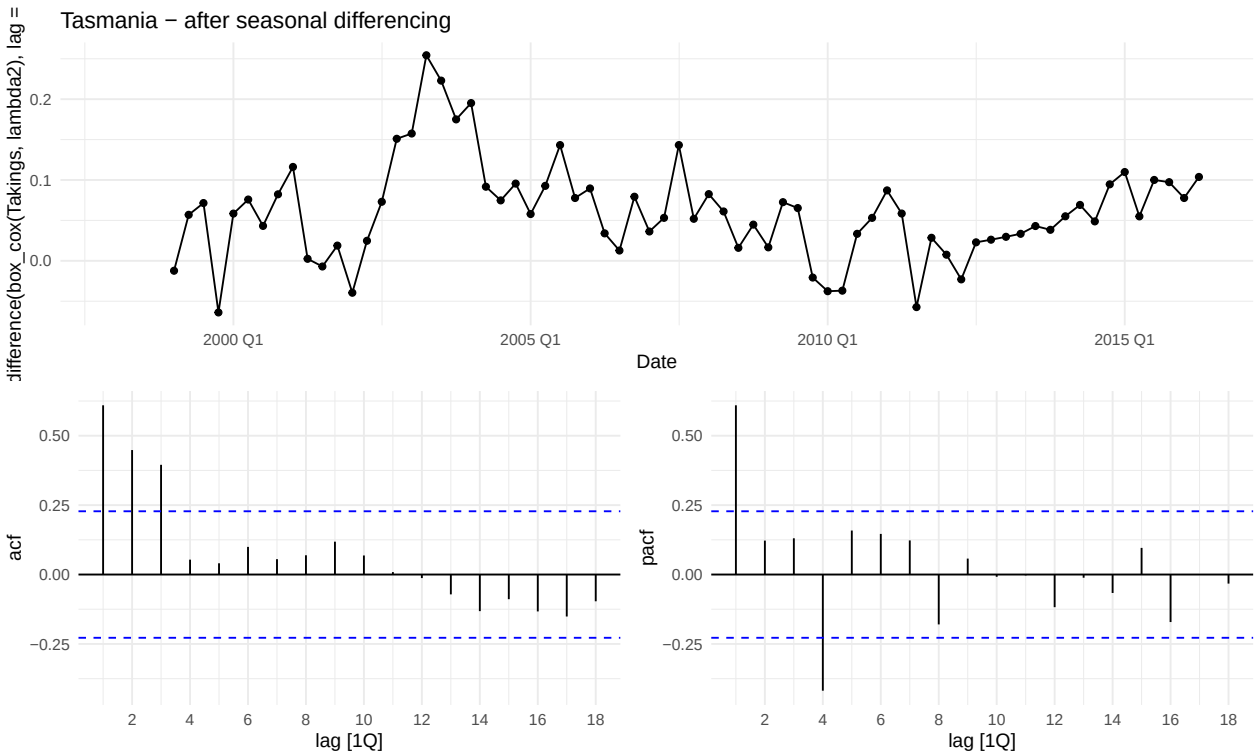
```
# Find optimal lambda
lambda2 <- tasmania |>
  features(Takings, features = guerrero) |>
  pull(lambda_guerrero)

lambda2
```

```
## [1] 0.001819643
```

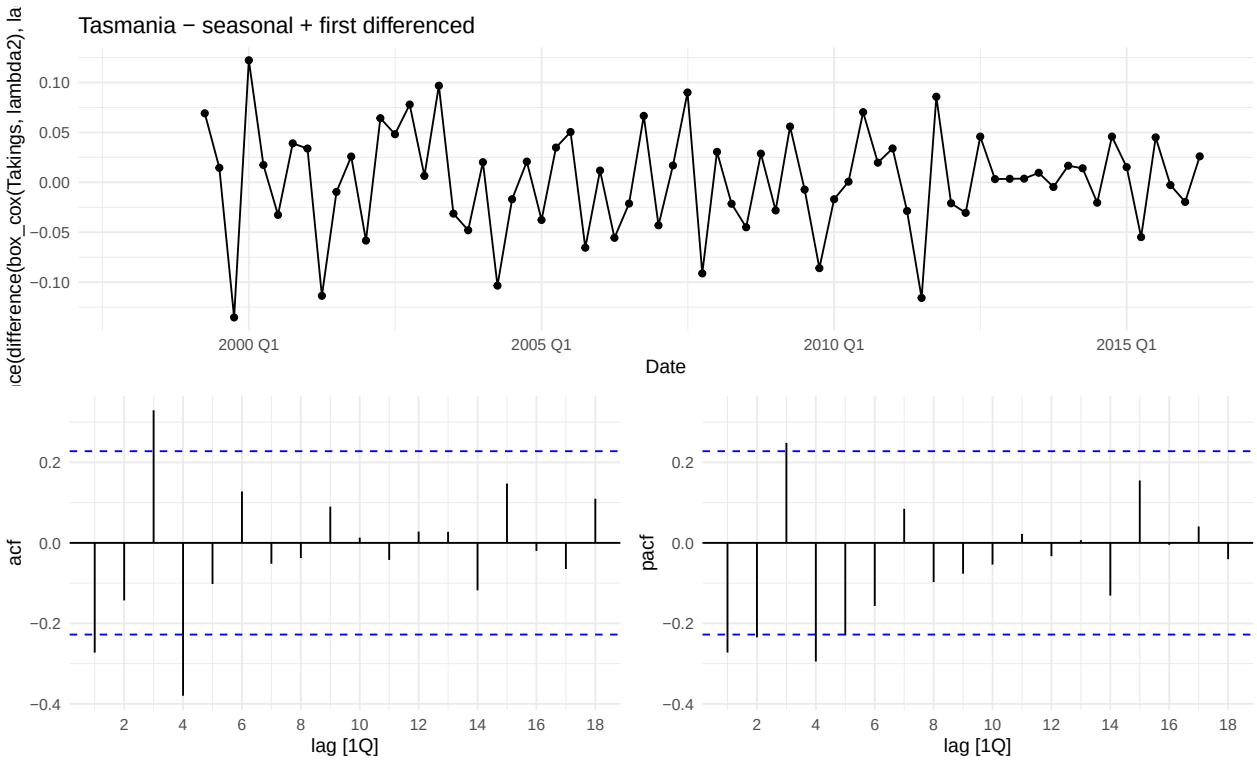
```
tasmania |> autoplot(box_cox(Takings, lambda2)) +
  labs(title = paste0("Tasmania - Box-Cox (  = ", round(lambda2,2), ")"))
```




```
# Seasonal difference first (period = 4 for quarterly)
tasmania |>
  gg_tsdisplay(
    difference(box_cox(Takings, lambda2), lag = 4),
    plot_type = "partial") +
  labs(title = "Tasmania - after seasonal differencing")
```




```r
# Then apply first difference to remove remaining trend
tasmania |>
  gg_tsdisplay(
    difference(difference(box_cox(Takings, lambda2), lag = 4), 1),
    plot_type = "partial") +
  labs(title = "Tasmania - seasonal + first differenced")
```



```r
# Confirm stationarity
tasmania |>
  mutate(tak_bc = box_cox(Takings, lambda2)) |>
  features(difference(difference(tak_bc, lag=4), 1), unitroot_kpss)
```

```
## # A tibble: 1 x 3
##   State    kpss_stat kpss_pvalue
##   <chr>        <dbl>       <dbl>
## 1 Tasmania    0.0474         0.1
```

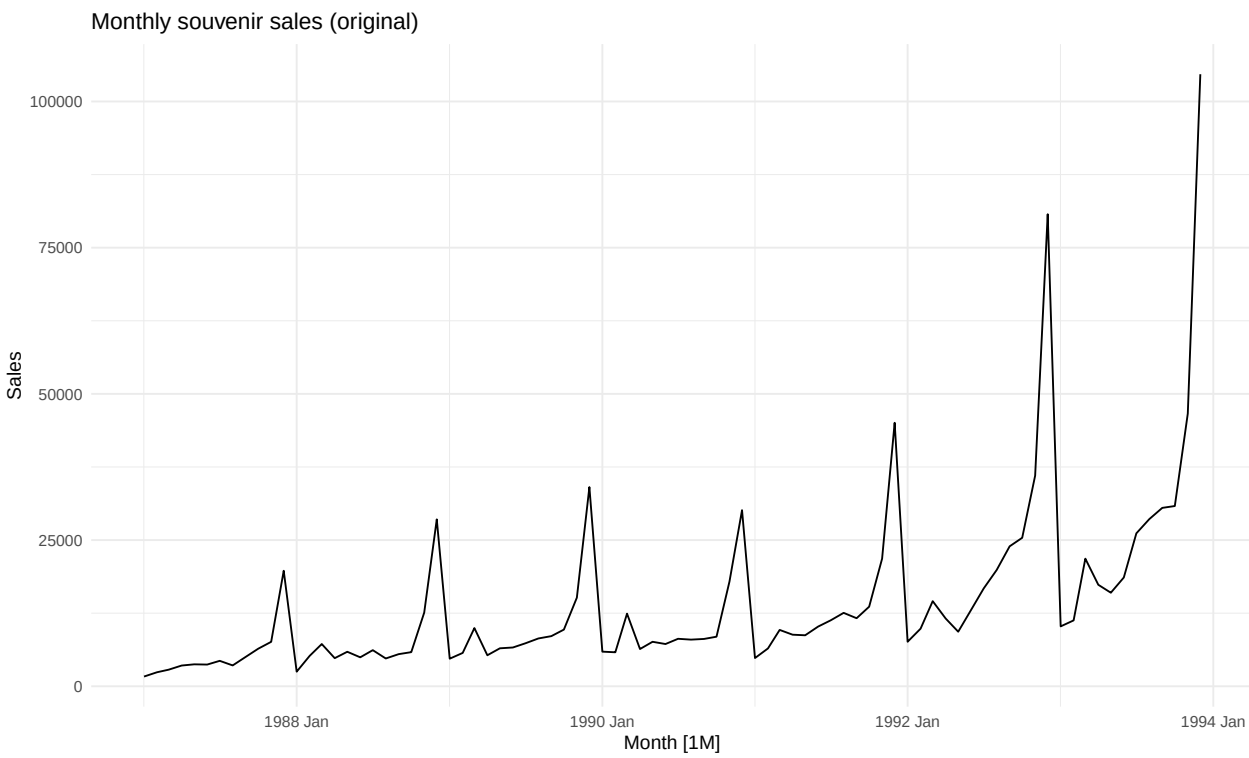
For Tasmania accommodation takings, the Guerrero method gives = 0.0018, which is essentially a log transformation, stabilising the growing seasonal variance.

After applying the Box-Cox transformation, seasonal differencing (D = 1, lag = 4)

and first differencing (d = 1), the KPSS test returns a p-value of 0.1 > 0.05, confirming the series is now stationary.

c. Series 3 — Monthly souvenir sales from souvenirs

```r
souvenirs |> autoplot(Sales) +
  labs(title = "Monthly souvenir sales (original)")
```

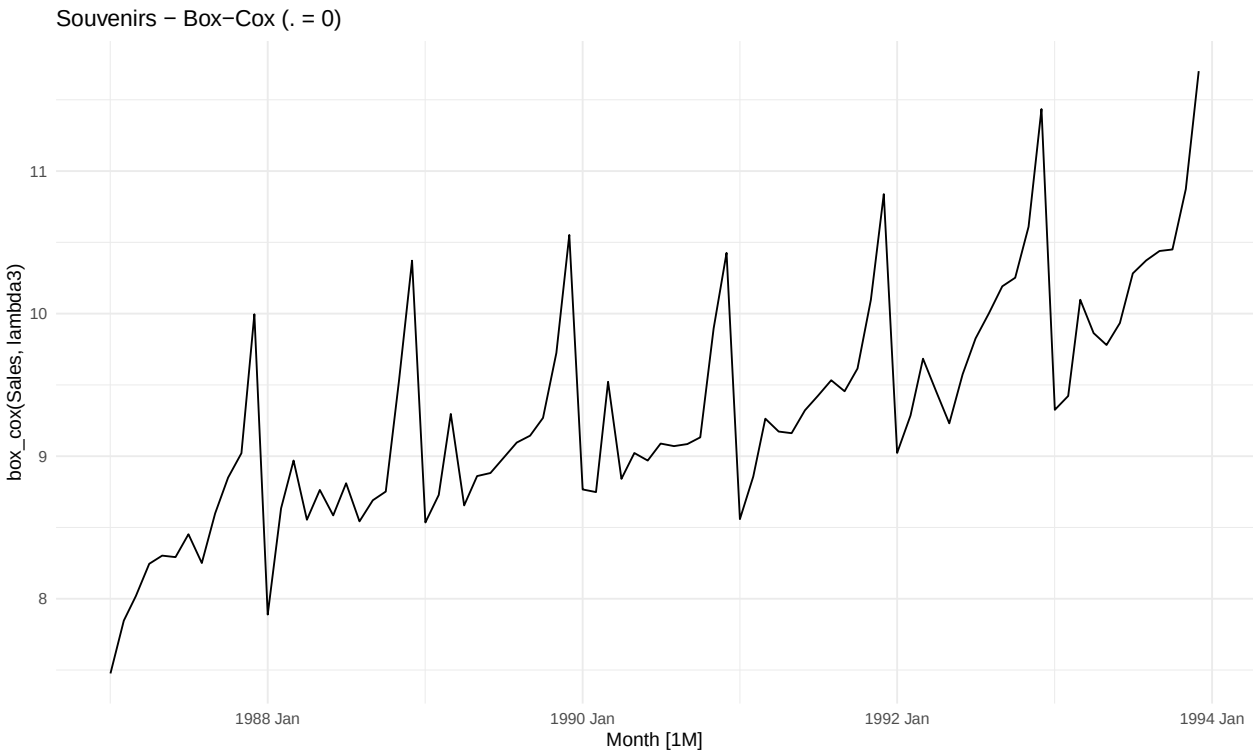


The series shows an upward trend, strong monthly seasonality (spikes every December — Christmas), AND the seasonal spikes grow enormously over time — classic multiplicative seasonality requiring variance stabilisation.

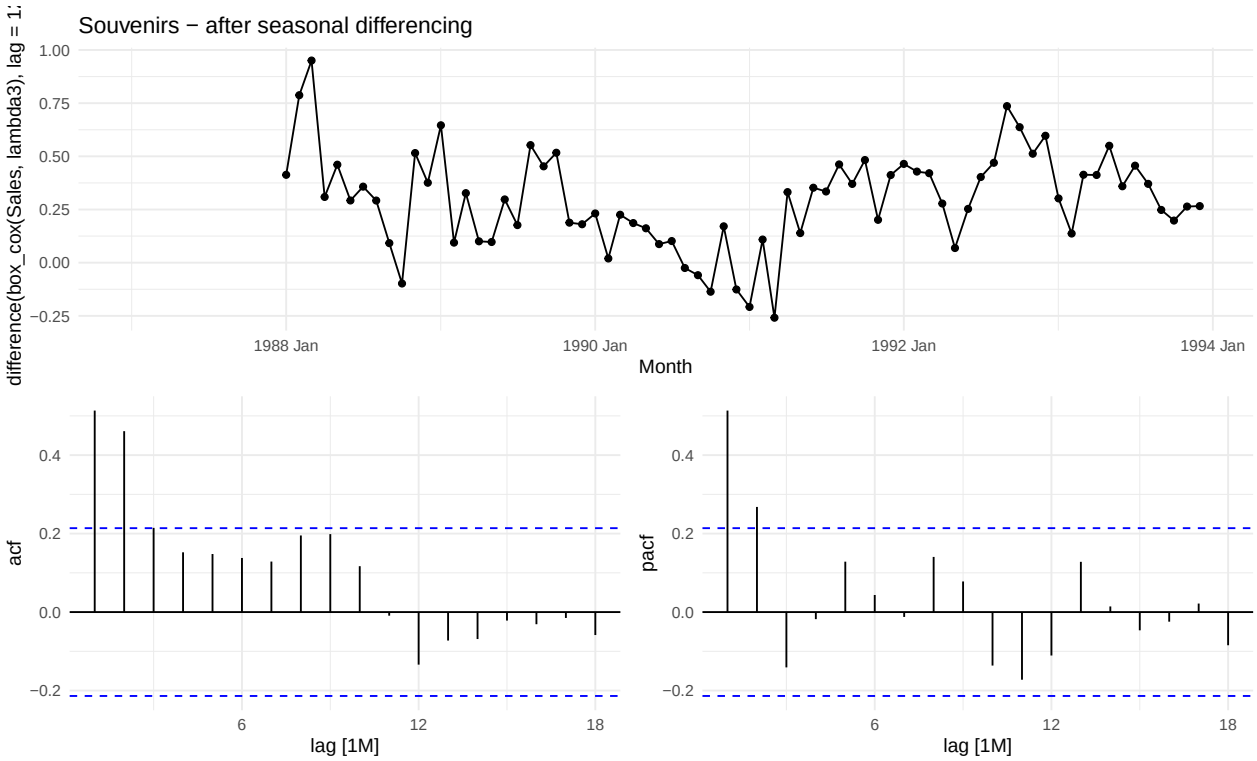
```r
# Find optimal lambda
lambda3 <- souvenirs |>
  features(Sales, features = guerrero) |>
  pull(lambda_guerrero)

lambda3 # expect close to 0 (log)
```

```
## [1] 0.002118221
```

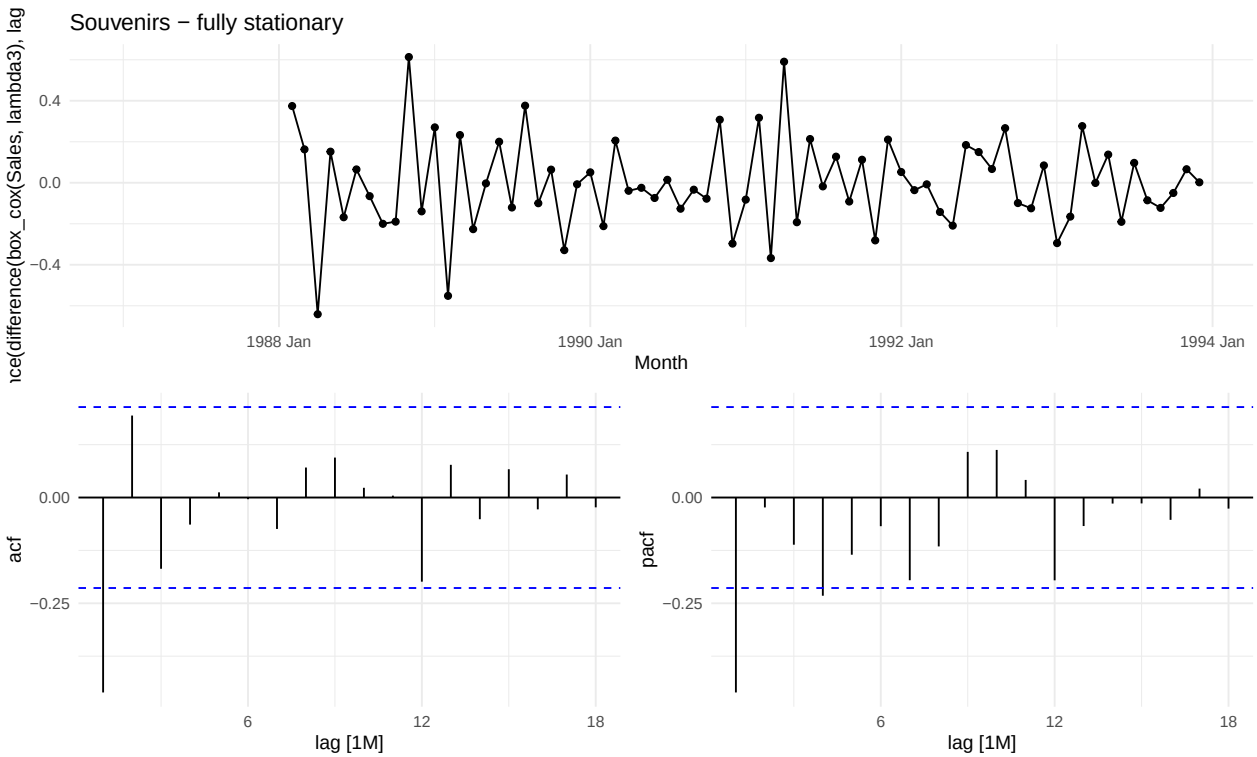
```r
souvenirs |> autoplot(box_cox(Sales, lambda3)) +
  labs(title = paste0("Souvenirs - Box-Cox (  = ", round(lambda3,2), ")"))
```




```r
# Seasonal difference (period = 12 for monthly)
souvenirs |>
  gg_tsdisplay(
    difference(box_cox(Sales, lambda3), lag = 12),
    plot_type = "partial") +
  labs(title = "Souvenirs - after seasonal differencing")
```




```r
# Apply first difference to remove remaining trend
souvenirs |>
  gg_tsdisplay(
    difference(difference(box_cox(Sales, lambda3), lag = 12), 1),
    plot_type = "partial") +
  labs(title = "Souvenirs - fully stationary")
```




```r
# Confirm stationarity
souvenirs |>
  mutate(sal_bc = box_cox(Sales, lambda3)) |>
  features(difference(difference(sal_bc, lag=12), 1), unitroot_kpss)
```

```
## # A tibble: 1 x 2
##   kpss_stat kpss_pvalue
##       <dbl>       <dbl>
## 1    0.0381         0.1
```

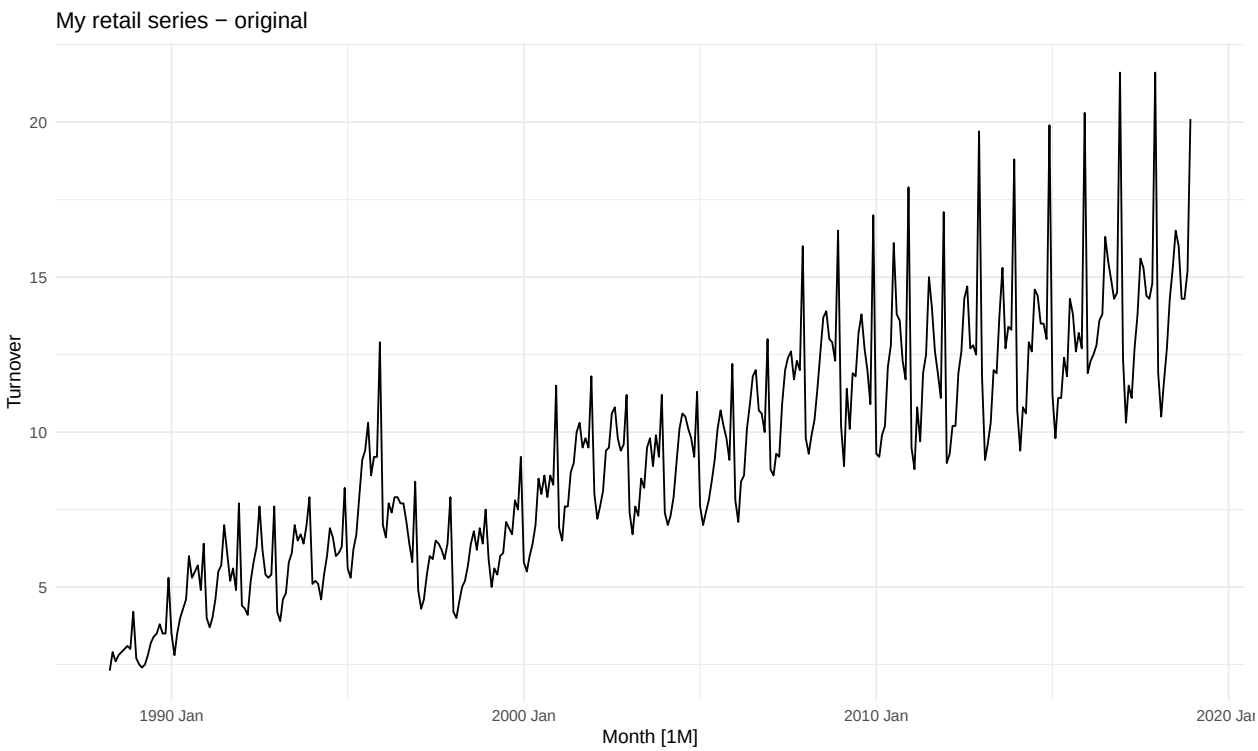
Souvenir sales:

The Guerrero method gives = 0.0021, again essentially a log transformation, which converts the multiplicative exponentially growing seasonality into an additive pattern. After Box-Cox transformation, seasonal differencing (D = 1, lag = 12) and first differencing (d = 1), the KPSS p-value = 0.1 > 0.05 confirms stationarity.

All three KPSS statistics are very small and all p-values hit the maximum reported value of 0.1 in R — this means the evidence for stationarity is very strong in all three cases. The transformations and differencing worked perfectly.

```r
set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))
# Plot the raw series
myseries |> autoplot(Turnover) +
  labs(title = "My retail series - original")
```

9.5 For your retail data (from Exercise 7 in Section 2.10), find the appropriate order of differencing (after transformation if necessary) to obtain stationary data.

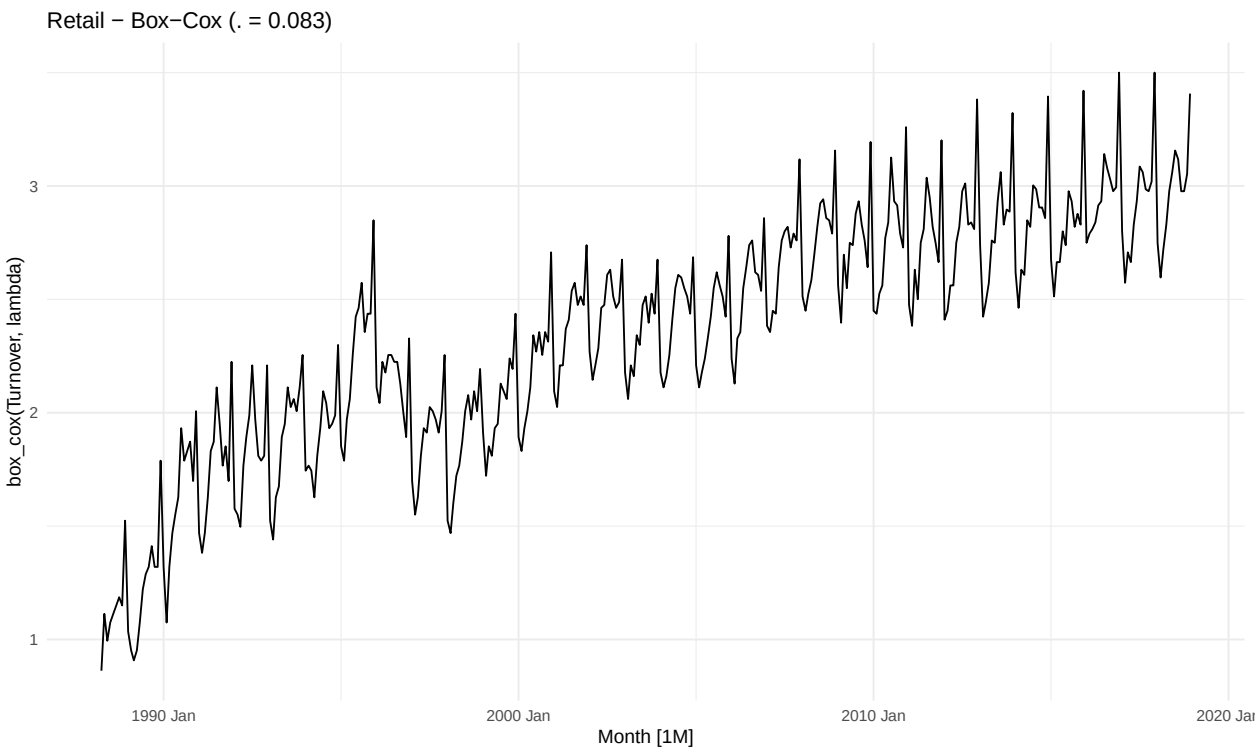


Strong upward trend. Strong Seasonality, amplifying over time.

```
# Guerrero method for optimal lambda
lambda <- myseries |>
  features(Turnover, features = guerrero) |>
  pull(lambda_guerrero)

lambda # print and share this value
```

```
## [1] 0.08303631
```

```
# Plot transformed series
myseries |> autoplot(box_cox(Turnover, lambda)) +
  labs(title = paste0("Retail - Box-Cox (  = ", round(lambda, 3), ")"))
```




```r
# Check if seasonal differencing is needed first
myseries |>
  features(box_cox(Turnover, lambda), unitroot_nsdiffs)
```

```
## # A tibble: 1 x 3
##   State              Industry                                              nsdiffs
##   <chr>              <chr>                                                   <int>
## 1 Northern Territory Clothing, footwear and personal accessory retailing       1
```

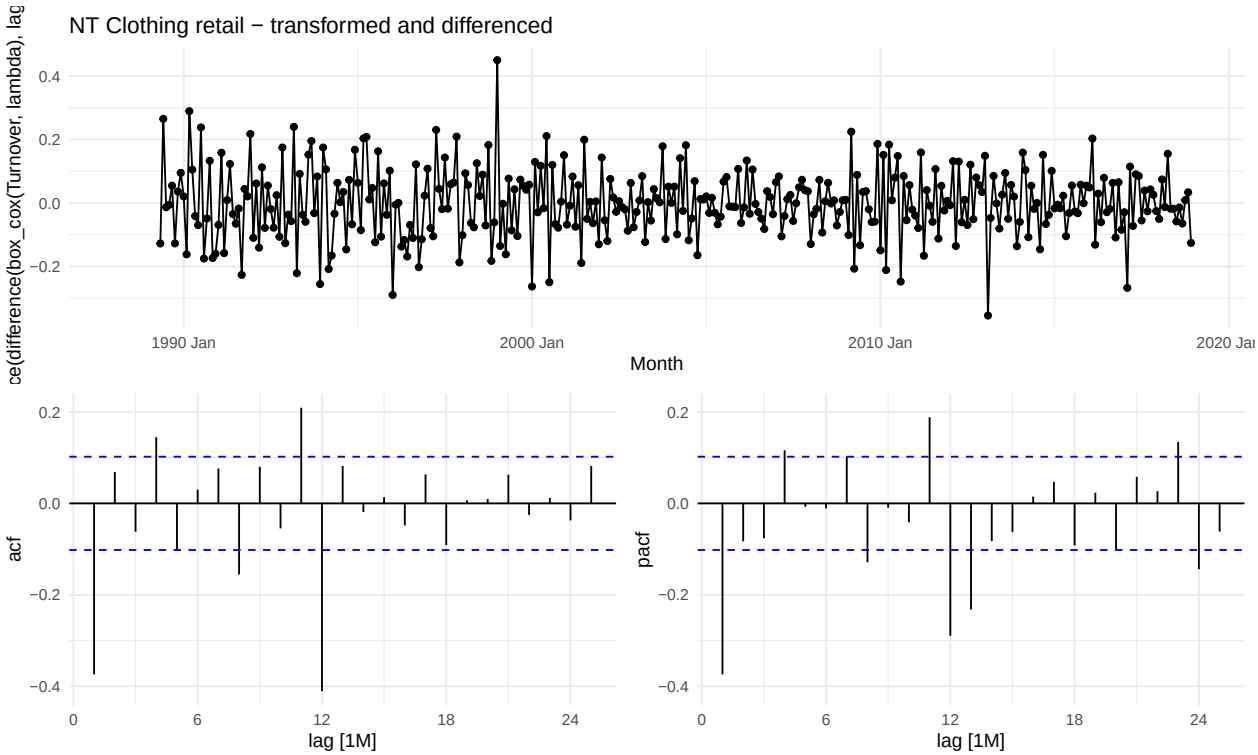
```r
# Then check ordinary differencing needed
myseries |>
  features(box_cox(Turnover, lambda), unitroot_ndiffs)
```

```
## # A tibble: 1 x 3
##   State              Industry                                              ndiffs
##   <chr>              <chr>                                                  <int>
## 1 Northern Territory Clothing, footwear and personal accessory retailing      1
```

lambda = 0.083 Extremely close to 0 → essentially a log transformation This means your retail series has growing variance over time — the seasonal swings get bigger as turnover increases. The log transformation will stabilise this.

nsdiffs Result = 1 Result Meaning nsdiffs = 1 Seasonal differencing IS needed → D = 1, lag = 12 (monthly data) This confirms the series has a strong repeating monthly seasonal pattern that must be removed before checking for ordinary differencing.

```r
# Apply seasonal diff (lag=12) then ordinary diff
myseries |>
  gg_tsdisplay(
    difference(difference(box_cox(Turnover, lambda), lag = 12), 1),
    plot_type = "partial") +
  labs(title = "NT Clothing retail - transformed and differenced")
```




```r
# KPSS confirmation
myseries |>
  features(
    difference(difference(box_cox(Turnover, lambda), lag = 12), 1),
    unitroot_kpss
  )
```

```
## # A tibble: 1 x 4
##   State              Industry                                kpss_stat kpss_pvalue
##   <chr>              <chr>                                       <dbl>       <dbl>
## 1 Northern Territory Clothing, footwear and personal acce~      0.0165         0.1
```

```r
# KPSS test on fully transformed + differenced series
myseries |>
  features(
    difference(difference(box_cox(Turnover, lambda), lag = 12), 1),
    unitroot_kpss
  )
```

```
## # A tibble: 1 x 4
##   State              Industry                                kpss_stat kpss_pvalue
##   <chr>              <chr>                                       <dbl>       <dbl>
## 1 Northern Territory Clothing, footwear and personal acce~      0.0165         0.1
```

- KPSS stats: KPSS statistic 0.01648 Extremely small
- p-value = 0.1 > 0.05 → Fail to reject null → Stationary confirmed

For the Northern Territory Clothing, footwear and personal accessory retailing series, the Guerrero method gives = 0.083, essentially a log transformation, which stabilises the growing seasonal variance. The series requires seasonal differencing (D = 1, lag = 12) to remove the monthly seasonal pattern, followed by first differencing (d = 1) to remove the trend. After transformation and differencing, the ACF shows only 5 small scattered spikes within ±0.2 — consistent with chance crossings rather than real structure. The KPSS test

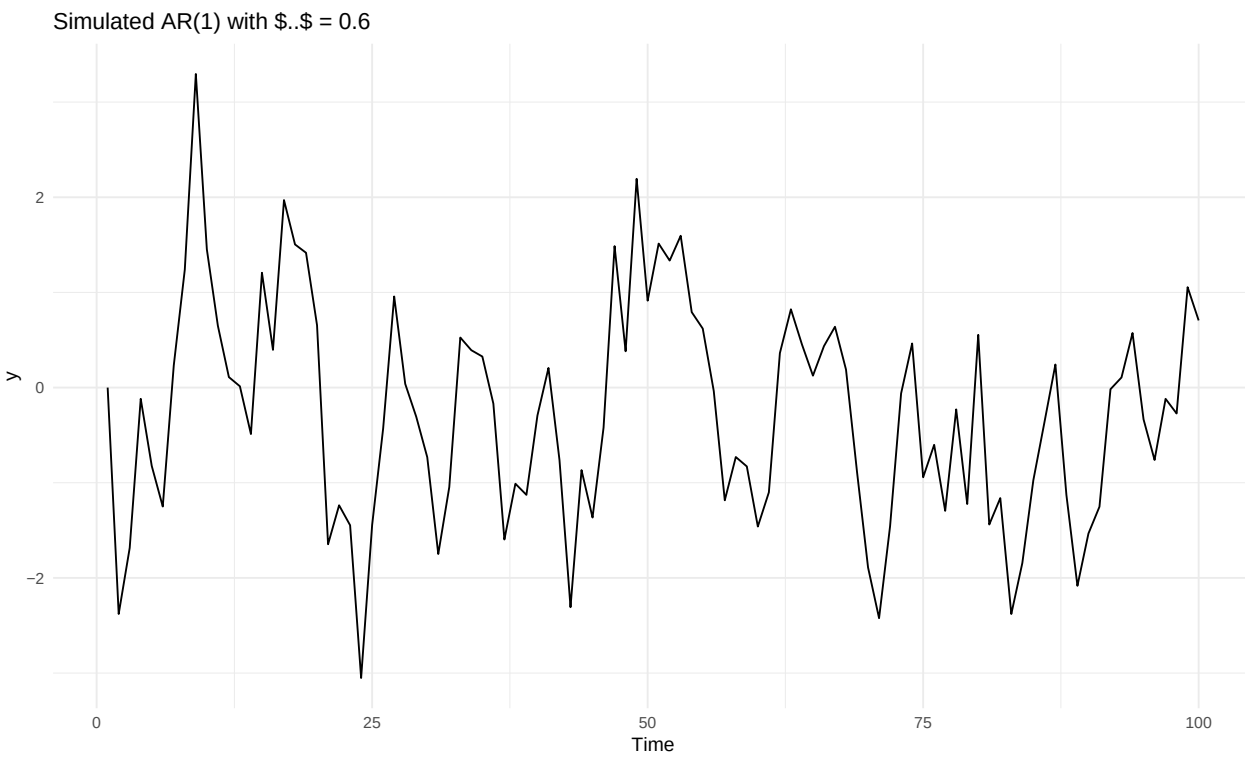
confirms stationarity with a statistic of 0.0165 and p-value = 0.1 > 0.05. Therefore the appropriate order is ARIMA(p, 1, q)(P, 1, Q)

9.6 Simulate and plot some data from simple ARIMA models. a. Use the following R code to generate data from an AR(1) model with 1=0.6 and 2=1 the process starts with y1=0

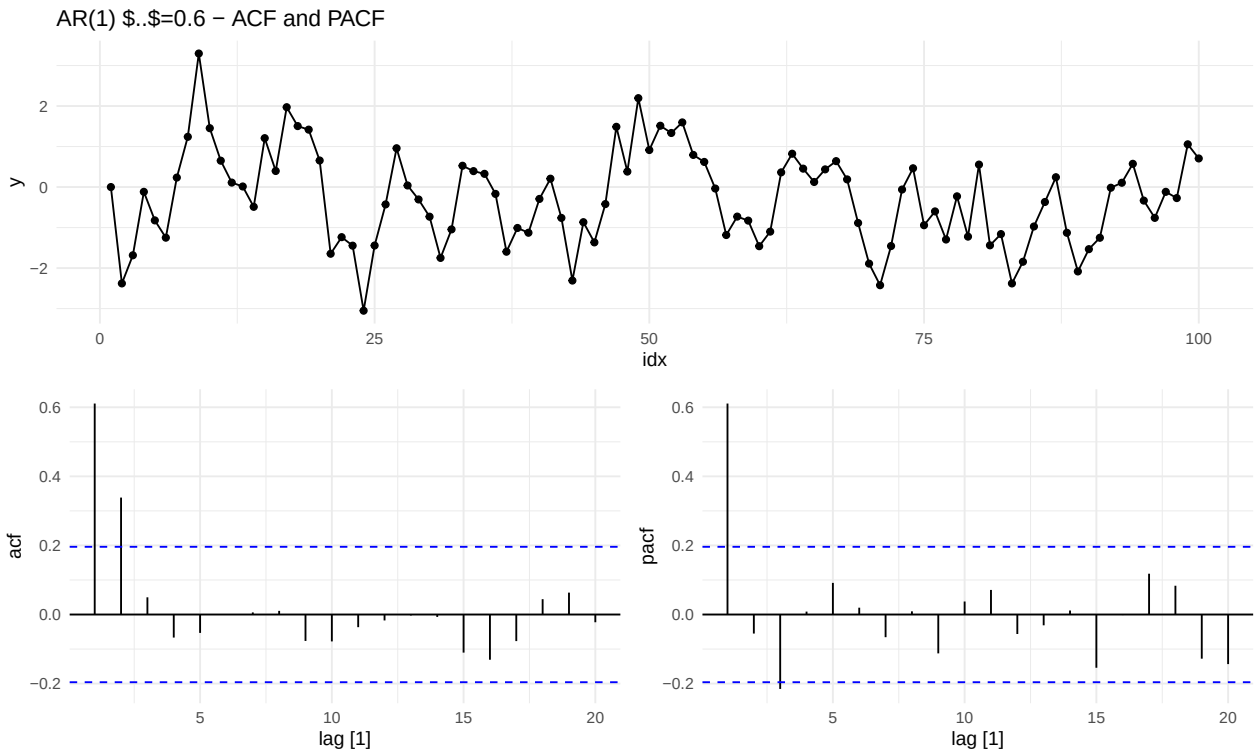
```
y <- numeric(100)       # create empty vector of 100 zeros
e <- rnorm(100)        # generate 100 white noise errors ~ N(0,1)
for(i in 2:100)       # loop from t=2 to t=100
  y[i] <- 0.6*y[i-1] + e[i]  # AR(1): y_t = 0.6*y_{t-1} + e_t

# y[1] = 0 by default (numeric() fills with zeros)
# Each new value = 0.6 × previous value + random shock

sim <- tsibble(idx = seq_len(100), y = y, index = idx)
# wraps the simulated data into a tsibble time series object
```

$ $ = 0.6 means the series has moderate positive autocorrelation — if today's value is high, tomorrow's is likely to be above zero too, but the effect fades over time. It does NOT drift or trend.

```
# Plot the time series
sim |> autoplot(y) +
  labs(title = "Simulated AR(1) with $ $ = 0.6",
       y = "y", x = "Time")
```




```
# Plot ACF and PACF together
sim |> gg_tsdisplay(y, plot_type = "partial") +
  labs(title = "AR(1) $ $=0.6 - ACF and PACF")
```



Time plot Fluctuates around zero with no trend. Smooth-ish waves due to $ $=0.6 memory - confirms stationarity

ACF plot Exponential decay starting from lag 1. Each spike 0.6k at lag k. Slowly tails off toward zero The negative spikes beyond the blue band at higher lags are due to sampling variability — you only have n = 100 observations. With such a small sample, a few random crossings are expected by chance. This is NOT a sign of real negative autocorrelation structure.

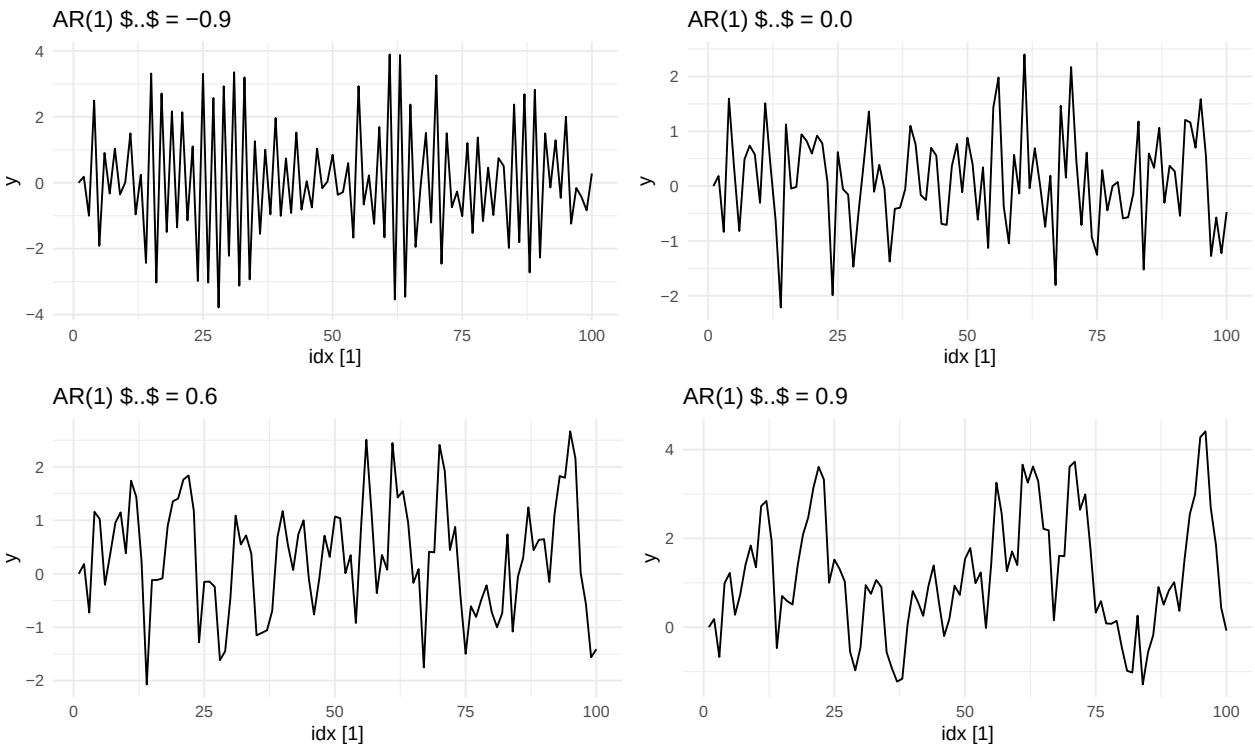
With 30 lags plotted, you expect 30 × 0.05 = 1.5 crossings by chance. A few crossings at lags 7+ in either direction is completely normal for n = 100.

PACF plot One single spike at lag 1 (0.6), then sharp cutoff — all other lags inside blue bands. This is the AR(1) signature

b. Produce a time plot for the series. How does the plot change as you change$ $1 ?

```
# Use same random errors for fair comparison
set.seed(1)
e <- rnorm(100)

# Function to simulate AR(1) for any phi
sim_ar1 <- function(phi, errors) {
  y <- numeric(100)
  for(i in 2:100)
    y[i] <- phi * y[i-1] + errors[i]
  tsibble(idx = seq_len(100), y = y, index = idx)
}

# Simulate 4 different phi values
sim1 <- sim_ar1(phi = -0.9, e)
sim2 <- sim_ar1(phi = 0.0, e)
sim3 <- sim_ar1(phi = 0.6, e)
```

```r
sim4 <- sim_ar1(phi = 0.9, e)

# Plot each one
p1 <- sim1 |> autoplot(y) + labs(title = "AR(1) $ $ = -0.9", y = "y")
p2 <- sim2 |> autoplot(y) + labs(title = "AR(1) $ $ = 0.0", y = "y")
p3 <- sim3 |> autoplot(y) + labs(title = "AR(1) $ $ = 0.6", y = "y")
p4 <- sim4 |> autoplot(y) + labs(title = "AR(1) $ $ = 0.9", y = "y")

# Combine into 2x2 grid
(p1 | p2) / (p3 | p4)
```



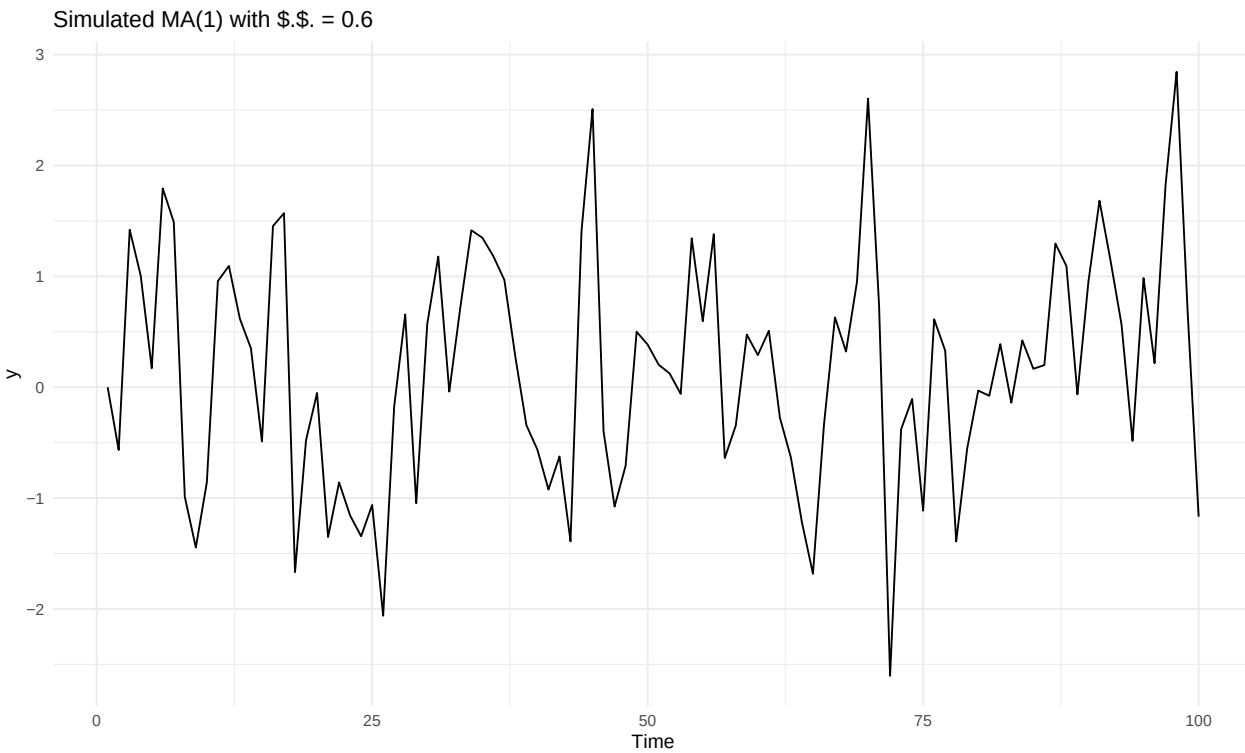
As φ_1 changes, the time plot changes dramatically.

- When $\varphi_1 = -0.9$, the series produces a rapid violent zigzag pattern alternating between ± 4 at every time step — strong negative autocorrelation means each value overshoots in the opposite direction.
- When $\varphi_1 = 0$, the series is pure white noise fluctuating randomly between ± 2 with no memory or pattern.
- When $\varphi_1 = 0.6$, the series shows moderate smooth waves between ± 2 — consecutive values are similar but the series regularly crosses zero.
- When $\varphi_1 = 0.9$, the series produces long slow persistent waves reaching up to $+4$ that stay on one side of zero for extended periods, closely resembling a non-stationary random walk. In general, as φ_1 increases from -1 toward $+1$, the series transitions from rapid oscillation $\rightarrow$ white noise $\rightarrow$ smooth waves $\rightarrow$ near random walk behaviour.

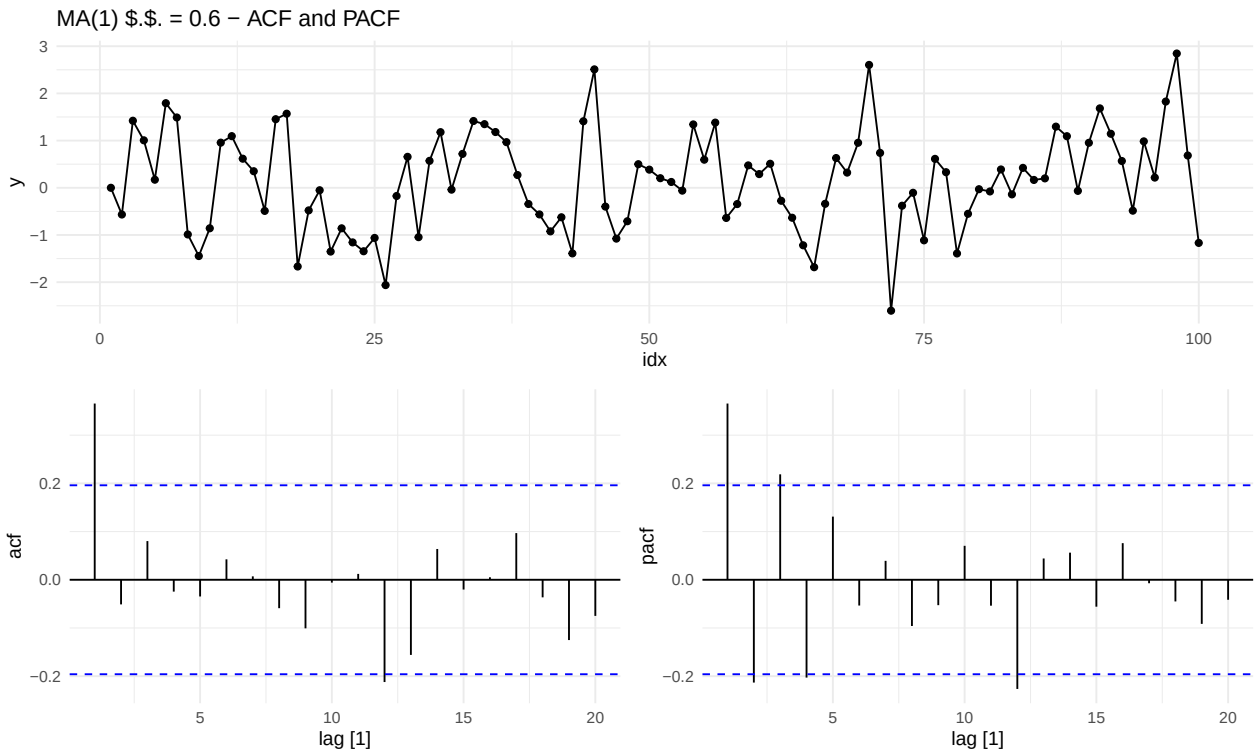
As φ_1 increases from 0 $\rightarrow$ 1, the plot becomes smoother and more persistent — waves get longer and slower. As φ_1 decreases from 0 $\rightarrow$ −1, the plot becomes more jagged and oscillating — it zigzags rapidly. The series is only stationary when $|\varphi_1| < 1$.

c. Write your own code to generate data from an MA(1) model with$\theta$$ 1 = 0.6 andσ 2 = 1

```r
# Simulate MA(1) with theta=0.6, sigma²=1
set.seed(123)
e <- rnorm(100)            # 100 white noise errors ~ N(0,1)
y <- numeric(100)          # empty vector of zeros

for(i in 2:100)
  y[i] <- e[i] + 0.6 * e[i-1]  # MA(1): y_t = e_t + 0.6*e_{t-1}

# Note: y[1] = e[1] + 0.6*e[0] but e[0] = 0 by default
# so y[1] = 0 (numeric() fills with zeros)

sim_ma <- tsibble(idx = seq_len(100), y = y, index = idx)

# Plot time series
sim_ma |> autoplot(y) +
  labs(title = "Simulated MA(1) with $ $  = 0.6", y = "y", x = "Time")
```

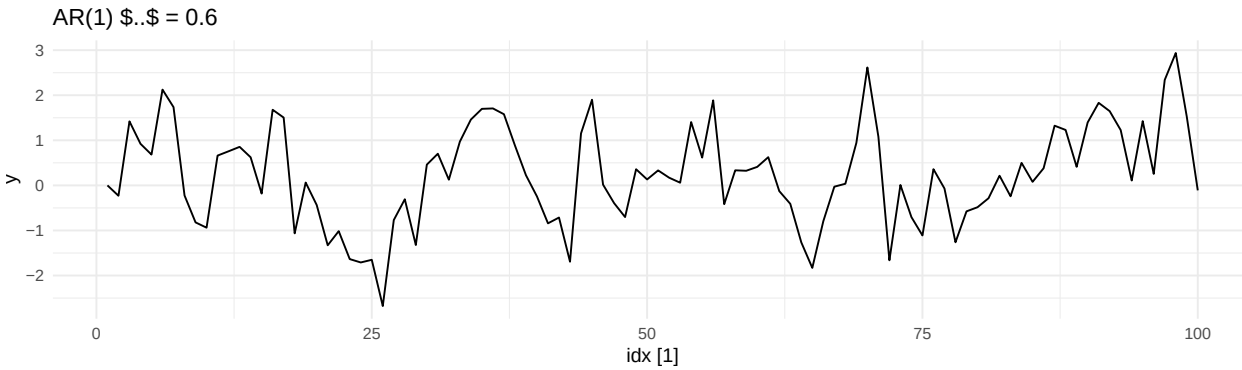


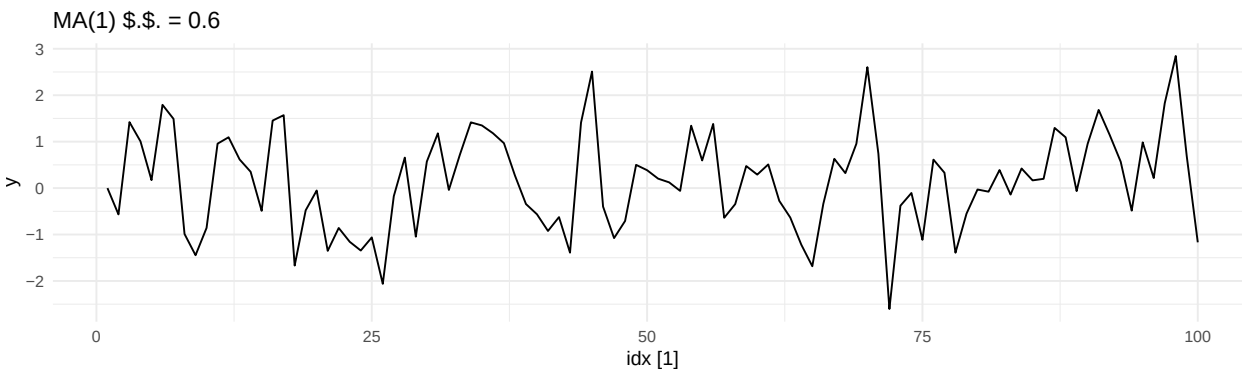

```r
# Plot ACF and PACF
sim_ma |> gg_tsdisplay(y, plot_type = "partial") +
  labs(title = "MA(1) $ $  = 0.6 - ACF and PACF")
```



- Unlike AR(1), the MA(1) process is always stationary regardless of θ .
- The time plot appears more jagged than AR(1) with $\varphi_1 = 0.6$ because memory only lasts one lag.
- The ACF shows one significant spike at lag 1 (0.44) then cuts off sharply to zero — the definitive signature of an MA(1) process.
- The PACF tails off gradually, which contrasts with AR(1) where the PACF cuts off and the ACF tails off.

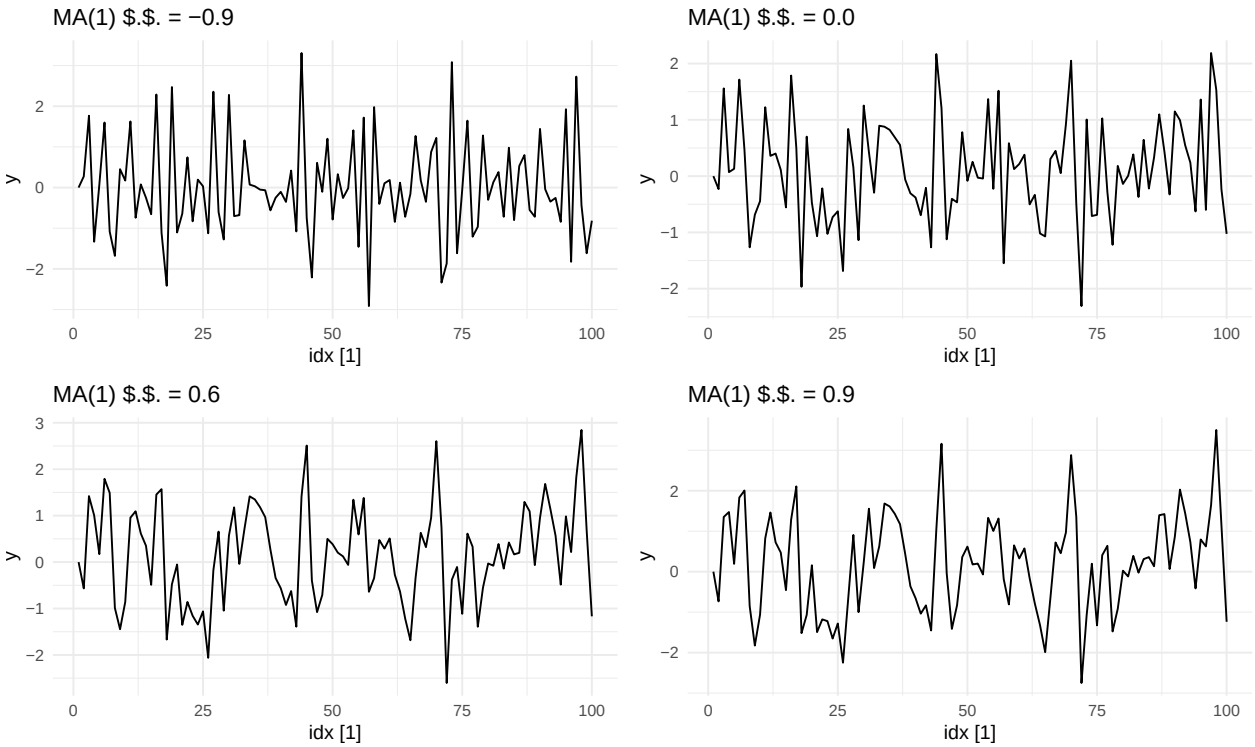
```r
# for quick comparison
set.seed(123)
e <- rnorm(100)

# AR(1) phi=0.6
y_ar <- numeric(100)
for(i in 2:100) y_ar[i] <- 0.6 * y_ar[i-1] + e[i]

# MA(1) theta=0.6
y_ma <- numeric(100)
for(i in 2:100) y_ma[i] <- e[i] + 0.6 * e[i-1]

# Combine into one tsibble for easy plotting
sim_ar <- tsibble(idx = seq_len(100), y = y_ar, index = idx)
sim_ma <- tsibble(idx = seq_len(100), y = y_ma, index = idx)

p_ar <- sim_ar |> autoplot(y) + labs(title = "AR(1) $ $ = 0.6")
p_ma <- sim_ma |> autoplot(y) + labs(title = "MA(1) $ $  = 0.6")

p_ar / p_ma  # stack vertically using patchwork
```





d. Produce a time plot for the series. How does the plot change as you change$\theta$$ 1 ?

```
# Same errors for fair comparison across all theta values
set.seed(123)
e <- rnorm(100)

# Function to simulate MA(1) for any theta
sim_ma1 <- function(theta, errors) {
  y <- numeric(100)
  for(i in 2:100)
    y[i] <- errors[i] + theta * errors[i-1]
  tsibble(idx = seq_len(100), y = y, index = idx)
}

# Simulate 4 different theta values
sim1 <- sim_ma1(theta = -0.9, e)
sim2 <- sim_ma1(theta = 0.0, e)
sim3 <- sim_ma1(theta = 0.6, e)
sim4 <- sim_ma1(theta = 0.9, e)

# Plot each one
p1 <- sim1 |> autoplot(y) + labs(title = "MA(1) $ $  = -0.9", y = "y")
p2 <- sim2 |> autoplot(y) + labs(title = "MA(1) $ $  = 0.0", y = "y")
p3 <- sim3 |> autoplot(y) + labs(title = "MA(1) $ $  = 0.6", y = "y")
p4 <- sim4 |> autoplot(y) + labs(title = "MA(1) $ $  = 0.9", y = "y")

# Combine into 2x2 grid
(p1 | p2) / (p3 | p4)
```



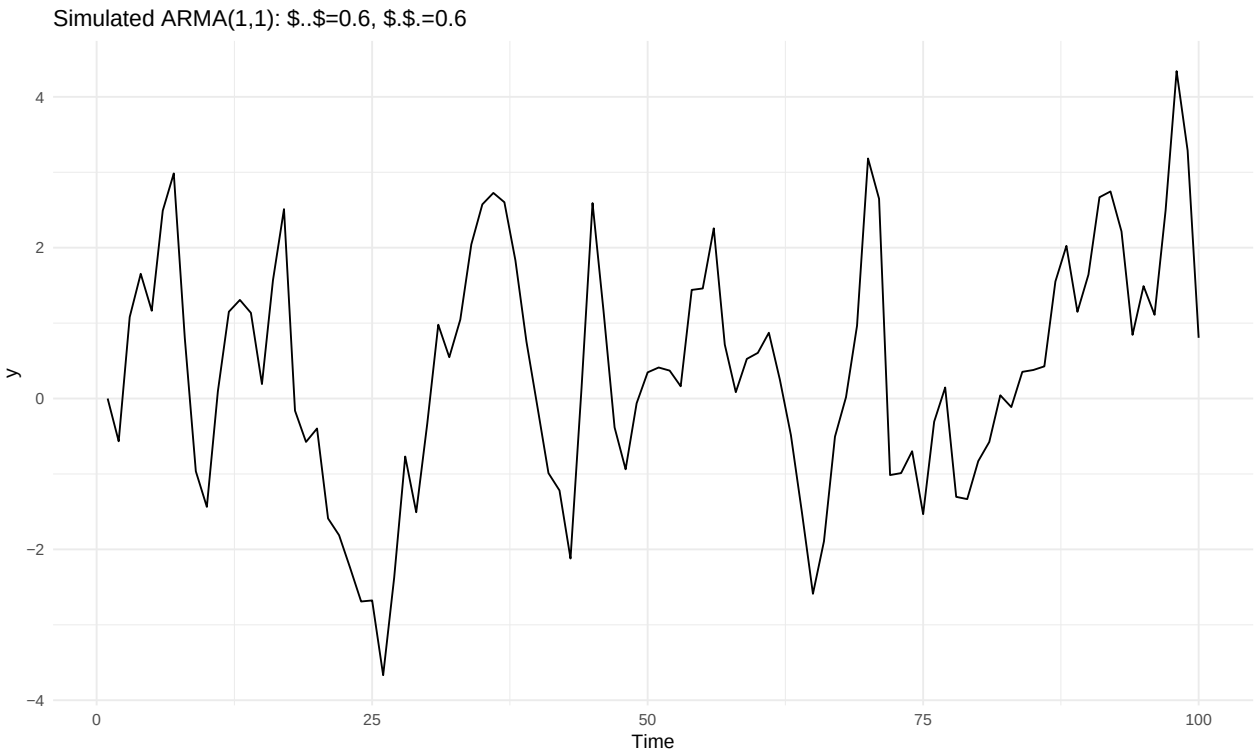
Interpretation: - As θ changes, the MA(1) time plot changes much more subtly than the equivalent AR(1) plots.

- When $\theta = -0.9$, the series shows a rapid jagged zigzag pattern between ± 3, as each shock pushes the next value in the opposite direction.
- When $\theta = 0.0$, the series is pure white noise with completely random fluctuations between ± 2 — no memory whatsoever.
- When $\theta = 0.6$, the series becomes slightly smoother as consecutive shocks reinforce each other mildly.
- When $\theta = 0.9$, the series is the smoothest of the four but still considerably more jagged than AR(1) with $\varphi_1 = 0.9$.

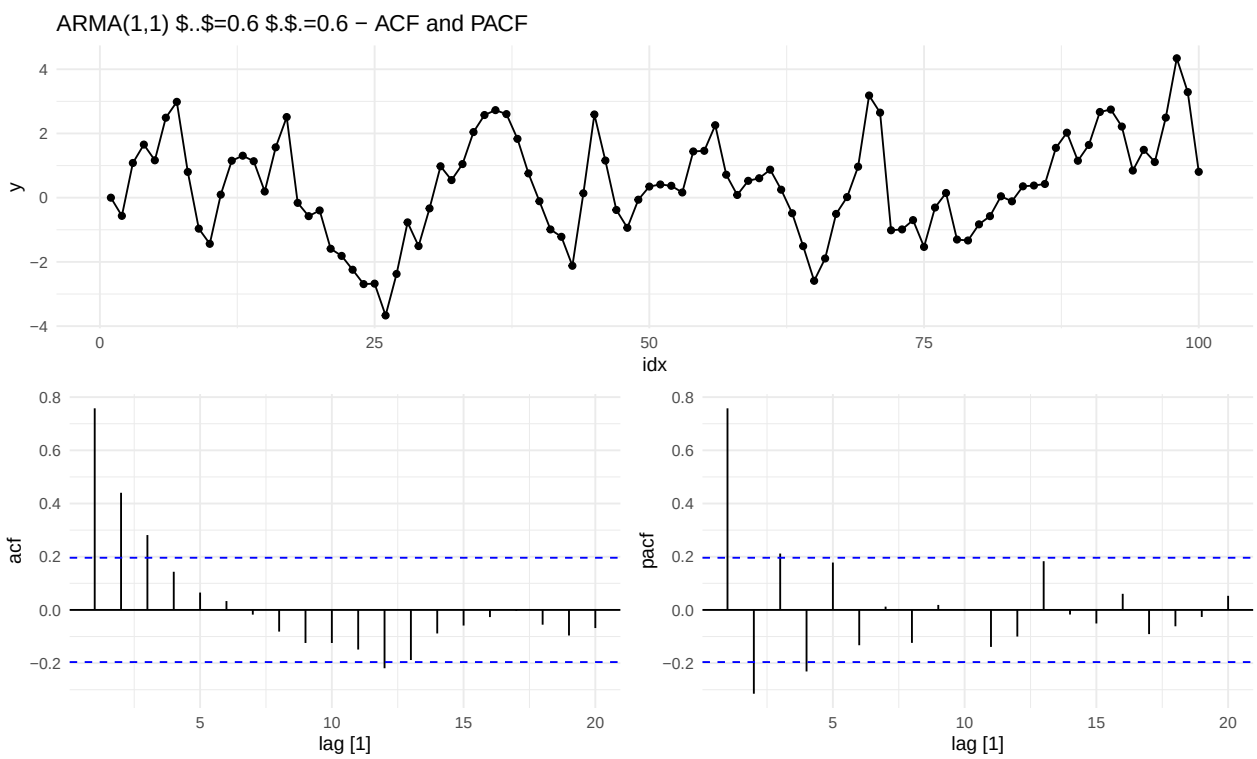
This is the fundamental difference between AR and MA processes — MA memory is strictly limited to q lags regardless of the size of θ, so no MA(1) series can ever exhibit the long persistent waves seen in AR(1) with large φ_1.

e. Generate data from an ARMA(1,1) model with$ $1 = 0.6$,$ $$ 1 = 0.6 andσ 2 = 1

```
set.seed(123)
e <- rnorm(100)            # 100 white noise errors ~ N(0,1)
y <- numeric(100)          # empty vector of zeros

# ARMA(1,1): y_t = phi*y_{t-1} + e_t + theta*e_{t-1}
for(i in 2:100)
  y[i] <- 0.6*y[i-1] + e[i] + 0.6*e[i-1]

sim_arma <- tsibble(idx = seq_len(100), y = y, index = idx)

# Plot time series
sim_arma |> autoplot(y) +
  labs(title = "Simulated ARMA(1,1): $ $=0.6, $ $ =0.6",
       y = "y", x = "Time")
```




```r
# Plot ACF and PACF
sim_arma |> gg_tsdisplay(y, plot_type = "partial") +
  labs(title = "ARMA(1,1) $ $=0.6 $ $ =0.6 - ACF and PACF")
```

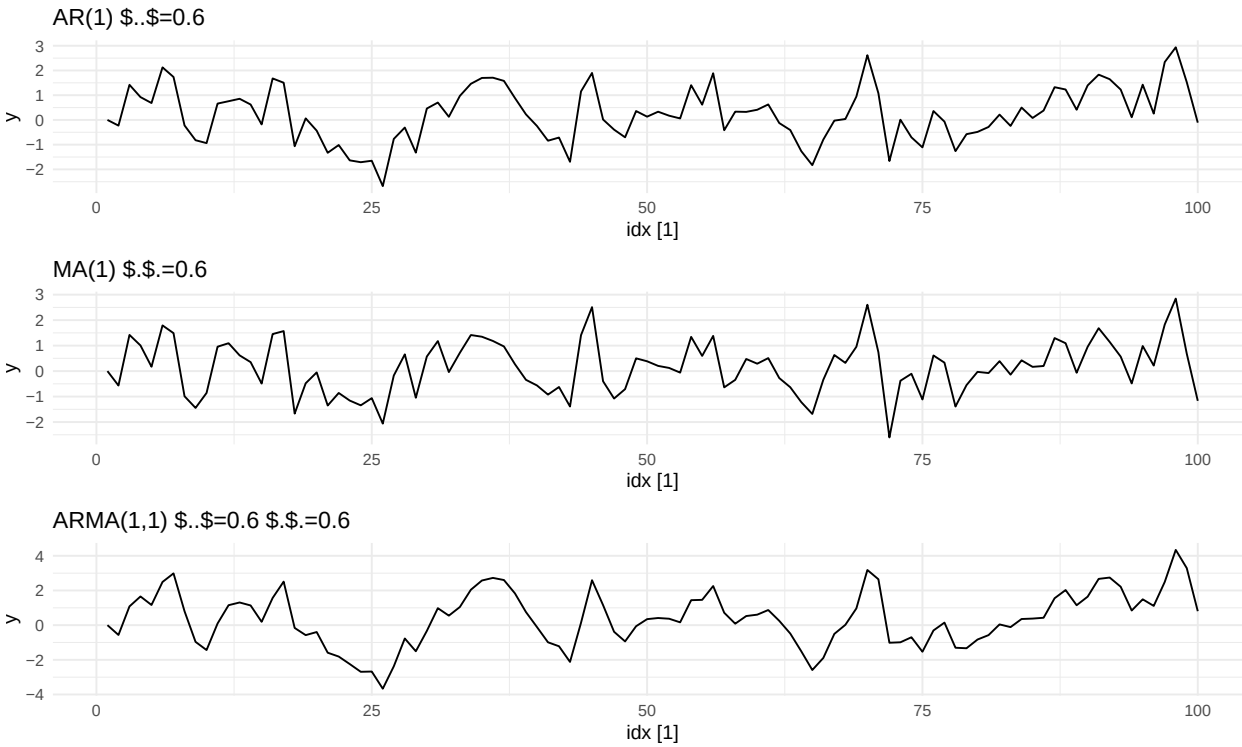


Interpretation

- The time plot is smoother and has wider range than either AR(1) or MA(1) alone because both past values and past errors reinforce each other.
- The ACF tails off gradually from lag 1 — unlike MA(1) which cuts off sharply.
- The PACF also tails off gradually — unlike AR(1) which cuts off sharply.
- When both ACF and PACF tail off without a clean cutoff, this is the diagnostic signature of a mixed ARMA process requiring both p and q terms.

Compare AR(1) vs MA(1) vs ARMA(1,1) — same errors

```r
set.seed(123)
e <- rnorm(100)

# AR(1) phi=0.6
y_ar <- numeric(100)
for(i in 2:100) y_ar[i] <- 0.6*y_ar[i-1] + e[i]

# MA(1) theta=0.6
y_ma <- numeric(100)
for(i in 2:100) y_ma[i] <- e[i] + 0.6*e[i-1]

# ARMA(1,1) phi=0.6, theta=0.6
y_arma <- numeric(100)
for(i in 2:100) y_arma[i] <- 0.6*y_arma[i-1] + e[i] + 0.6*e[i-1]

# Build tsibbles
s_ar <- tsibble(idx=seq_len(100), y=y_ar, index=idx)
s_ma <- tsibble(idx=seq_len(100), y=y_ma, index=idx)
s_arma <- tsibble(idx=seq_len(100), y=y_arma, index=idx)

# Plot all three stacked
p1 <- s_ar |> autoplot(y) + labs(title = "AR(1) $ $=0.6")
p2 <- s_ma |> autoplot(y) + labs(title = "MA(1) $ $ =0.6")
p3 <- s_arma |> autoplot(y) + labs(title = "ARMA(1,1) $ $=0.6 $ $ =0.6")

p1 / p2 / p3
```

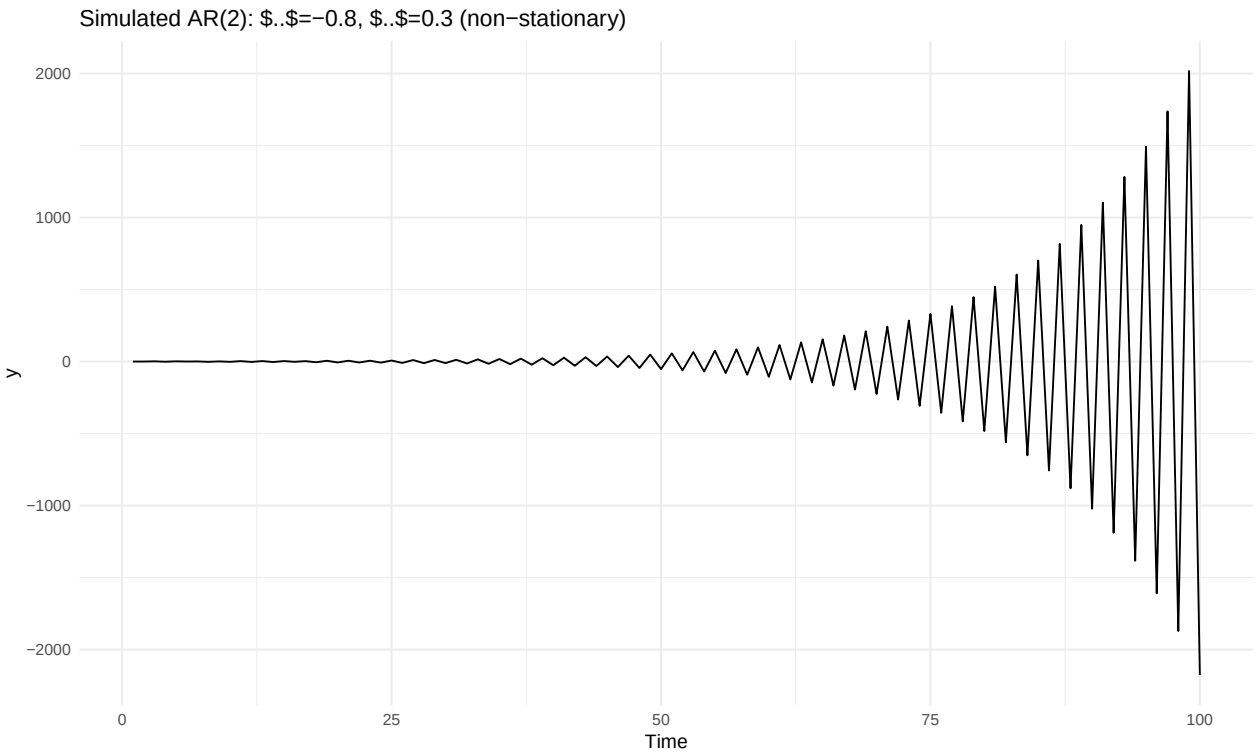


The ARMA(1,1) series smoother and have larger variance than either AR(1) or MA(1) alone — because both sources of memory (past values AND past errors) are reinforcing each other simultaneously with the same positive coefficient 0.6.

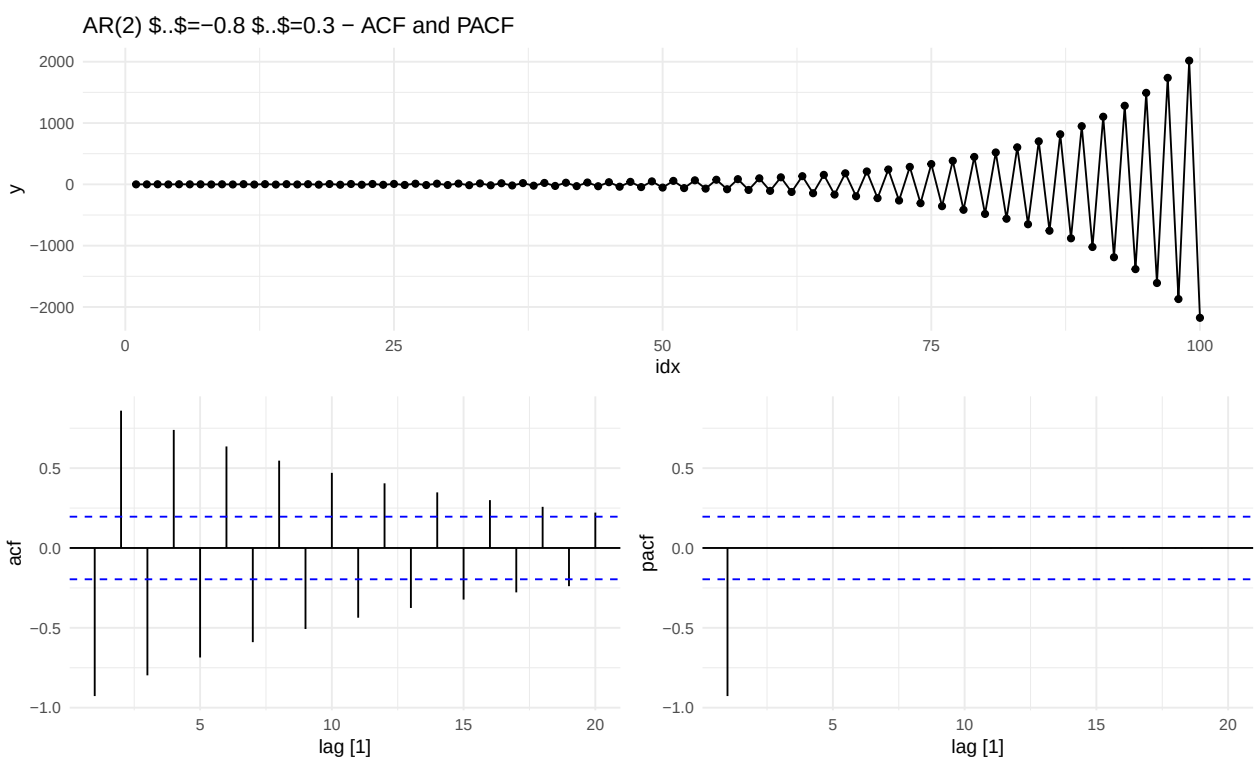
The ARMA(1,1) model yt=0.6yt−1+ t+0.6 t−1y_t = 0.6y_{t-1} + *t + 0.6* {t-1} yt=0.6yt−1+ t+0.6 t−1 combines both AR and MA components.

f. Generate data from an AR(2) model with$ $1 = -0.8,\ 2 = 0.3$*and* **$ 2 = 1 . (Note that these parameters will give a non-stationary series.)**

```
set.seed(123)
e <- rnorm(100)            # 100 white noise errors ~ N(0,1)
y <- numeric(100)          # empty vector - y[1]=y[2]=0

# AR(2): y_t = -0.8*y_{t-1} + 0.3*y_{t-2} + e_t
# Loop starts at i=3 because AR(2) needs 2 past values
for(i in 3:100)
  y[i] <- -0.8*y[i-1] + 0.3*y[i-2] + e[i]

sim_ar2 <- tsibble(idx = seq_len(100), y = y, index = idx)

# Plot time series
sim_ar2 |> autoplot(y) +
  labs(title = "Simulated AR(2): $ $=-0.8, $ $=0.3 (non-stationary)",
       y = "y", x = "Time")
```




```r
# Plot ACF and PACF
sim_ar2 |> gg_tsdisplay(y, plot_type = "partial") +
  labs(title = "AR(2) $ $=-0.8 $ $=0.3 - ACF and PACF")
```



The AR(2) model $y_t = -0.8y_{t-1} + 0.3y_{t-2} + \varepsilon_t$ is non-stationary because it violates the stationarity triangle condition: $\phi_2 - \phi_1 < 1$ gives $0.3 - (-0.8) = 1.1 > 1$.

Expected characteristics:

Time plot: Oscillatory behavior with explosive amplitude (not fluctuating around a constant mean)

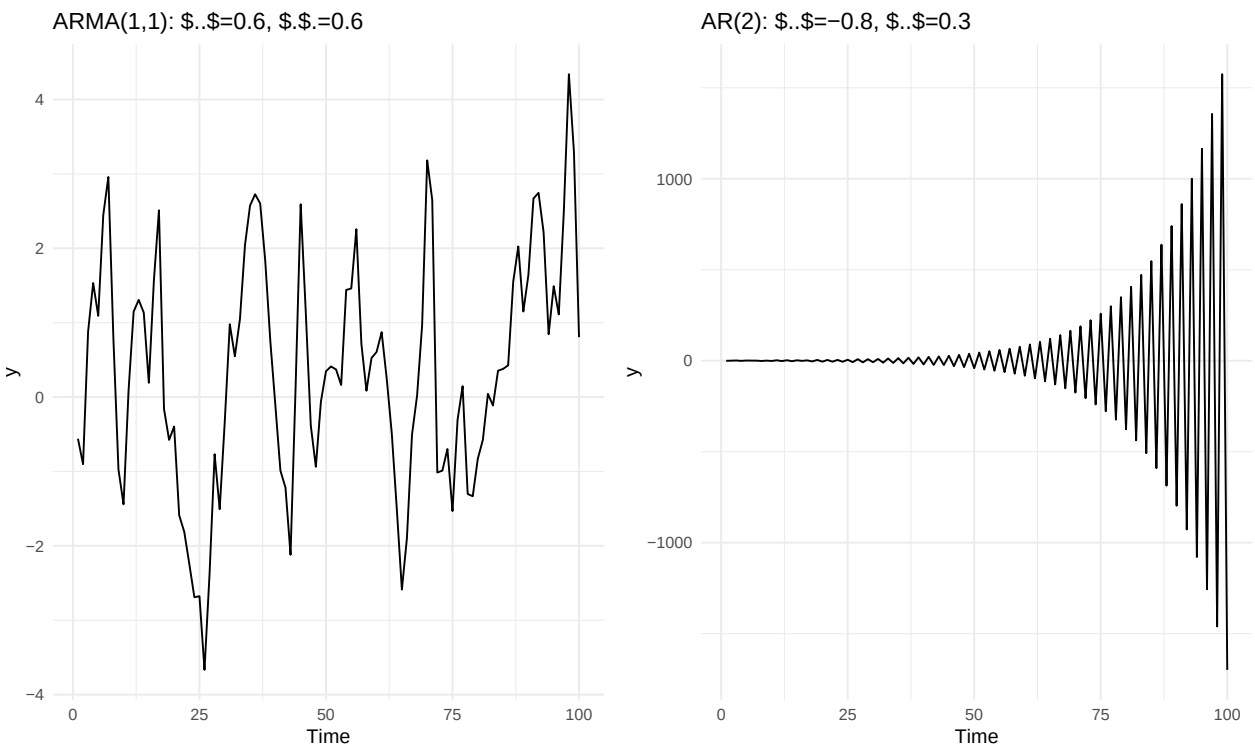
ACF: Very slow decay (characteristic of non-stationarity)

PACF: Two significant spikes at lags 1 and 2 (AR(2) signature)

The negative ϕ_1 creates alternating oscillations, while ϕ_2 prevents mean reversion, causing growing amplitude.

g. Graph the latter two series and compare them.

```
# ARMA(1,1) model with $ $=0.6, $ $ =0.6, $ ²$=1
set.seed(123)
n <- 100
e <- rnorm(n, mean = 0, sd = 1)
y_arma11 <- numeric(n)
y_arma11[1] <- e[1] # Initialize
for(i in 2:n) {
  y_arma11[i] <- 0.6 * y_arma11[i-1] + e[i] + 0.6 * e[i-1]
}

# AR(2) model with $ $=-0.8, $ $=0.3, $ ²$=1 (potentially non-stationary)
set.seed(123) # Same seed for comparison
e2 <- rnorm(n, mean = 0, sd = 1)
y_ar2 <- numeric(n)
y_ar2[1] <- e2[1]
y_ar2[2] <- -0.8 * y_ar2[1] + e2[2]
for(i in 3:n) {
  y_ar2[i] <- -0.8 * y_ar2[i-1] + 0.3 * y_ar2[i-2] + e2[i]
}

# Create tsibbles
sim_arma11 <- tsibble(idx = seq_len(n), y = y_arma11, index = idx)
sim_ar2 <- tsibble(idx = seq_len(n), y = y_ar2, index = idx)

# Plot both series
p1 <- sim_arma11 %>%
  autoplot(y) +
  labs(title = "ARMA(1,1): $ $=0.6, $ $ =0.6",
       y = "y", x = "Time")

p2 <- sim_ar2 %>%
  autoplot(y) +
  labs(title = "AR(2): $ $=-0.8, $ $=0.3",
       y = "y", x = "Time")

# Display side by side
p1 + p2
```




```r
# Quick comparison
arma11_summary <- sim_arma11 %>%
  features(y, list(mean = mean, sd = sd))

ar2_summary <- sim_ar2 %>%
  features(y, list(mean = mean, sd = sd))

print("ARMA(1,1) summary:")
```

```
## [1] "ARMA(1,1) summary:"
```

```r
print(arma11_summary)
```

```
## # A tibble: 1 x 2
##    mean    sd
##   <dbl> <dbl>
## 1 0.365  1.56
```

```r
print("AR(2) summary:")
```

```
## [1] "AR(2) summary:"
```

```r
print(ar2_summary)
```

```
## # A tibble: 1 x 2
##    mean    sd
##   <dbl> <dbl>
## 1 -8.75  456.
```

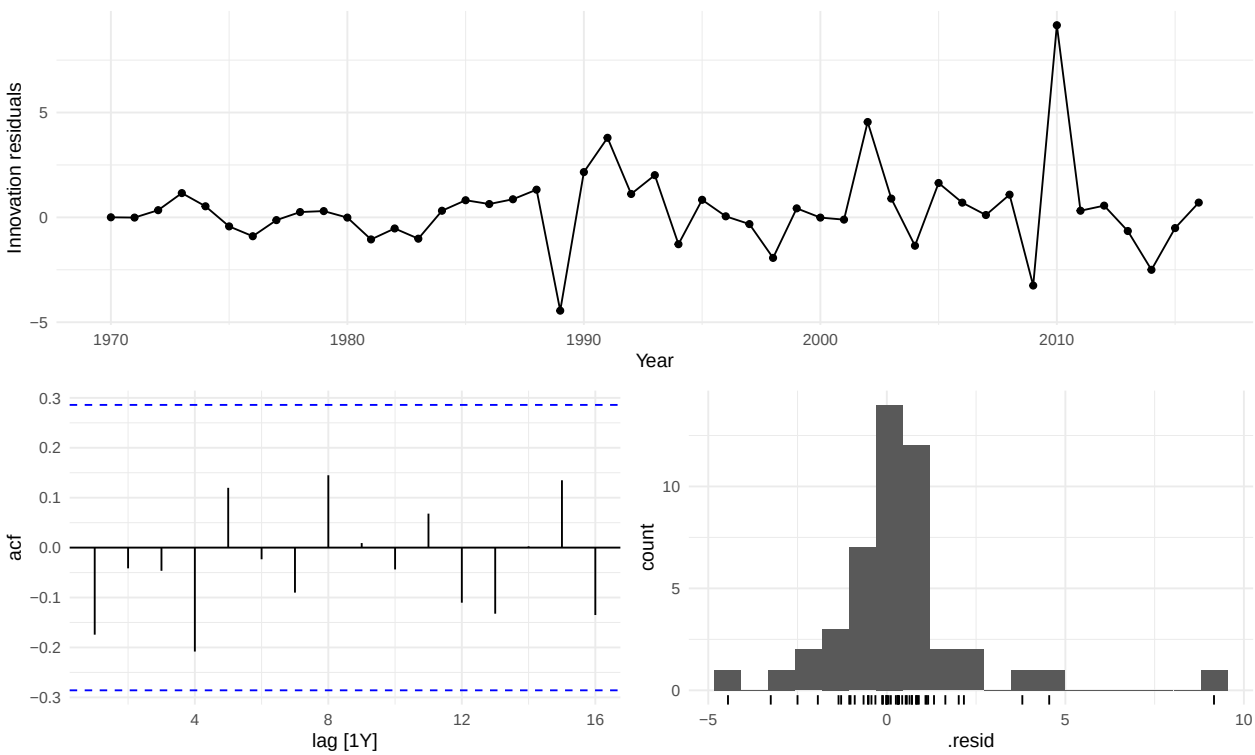
- ARMA(1,1): Stationary series with mean near zero (0.365) and stable SD (1.56).
- AR(2): Non-stationary series with explosive behavior—mean far from zero (-8.75) and huge SD (456), confirming the parameters produce a non-stationary process as expected.

9.7 Consider aus_airpassengers, the total number of passengers (in millions) from Australian air carriers for the period 1970-2011. a. Use ARIMA() to find an appropriate ARIMA model. What model was selected. Check that the residuals look like white noise. Plot forecasts for the next 10 periods.

```r
# Load data and find ARIMA model
aus_airpassengers <- aus_airpassengers

# Fit ARIMA model automatically
fit <- aus_airpassengers %>%
  model(ARIMA(Passengers))

# View the selected model
report(fit)
```

```
## Series: Passengers
## Model: ARIMA(0,2,1)
##
## Coefficients:
##           ma1
##       -0.8963
## s.e.   0.0594
##
## sigma^2 estimated as 4.308:  log likelihood=-97.02
## AIC=198.04   AICc=198.32   BIC=201.65
```

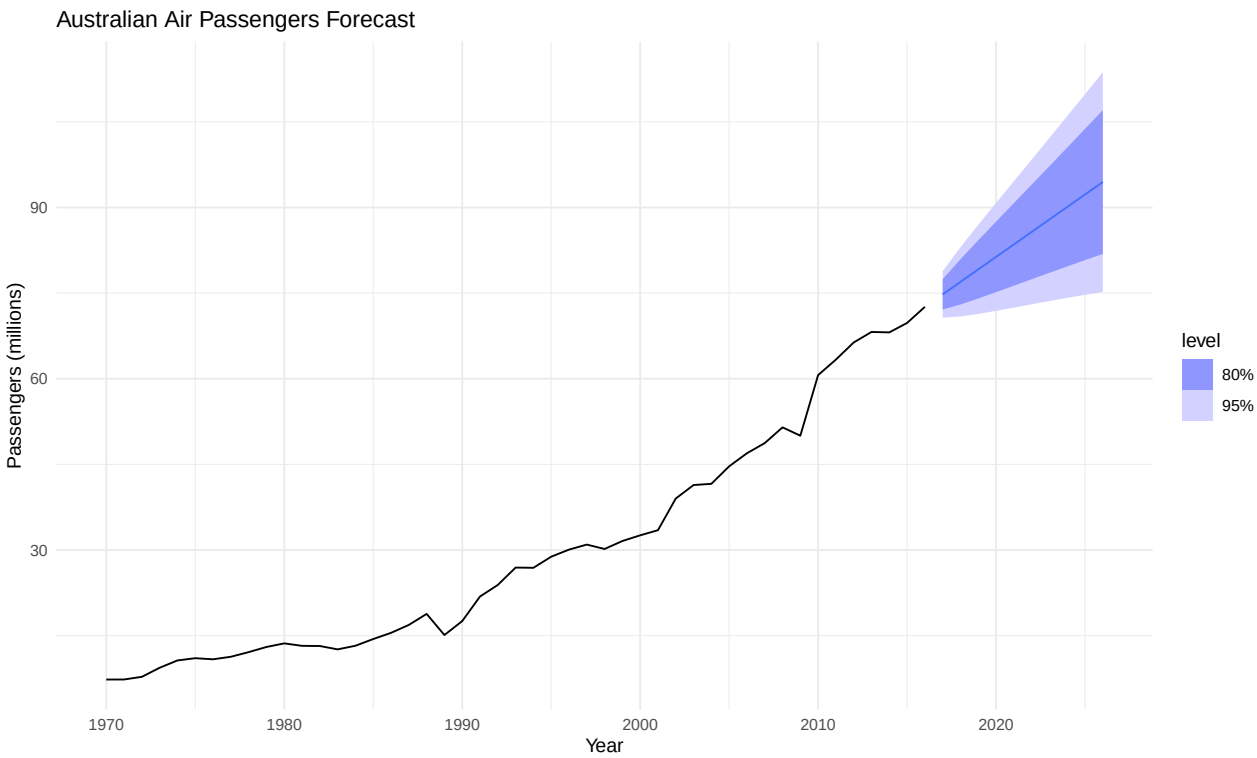
```r
# Check residuals
fit %>% ggtime::gg_tsresiduals()
```




```r
# Ljung-Box test for white noise
fit %>% augment() %>%
  features(.innov, ljung_box, lag = 10)
```

```
## # A tibble: 1 x 3
##   .model            lb_stat lb_pvalue
##   <chr>               <dbl>     <dbl>
## 1 ARIMA(Passengers)    6.70     0.754
```

```r
# Generate forecasts for next 10 periods
fit %>%
  forecast(h = 10) %>%
  autoplot(aus_airpassengers) +
  labs(title = "Australian Air Passengers Forecast",
       y = "Passengers (millions)")
```



Model interpretation:

- ARIMA(0,2,1) means: 2nd order differencing with one MA term
- MA coefficient: -0.8963 (highly negative, significant)
- Most residuals within ±5 (one outlier around 8-9)
- ACF within blue bands → residuals appear white noise
- Forecast will show increasing trend with widening prediction intervals
- The model is appropriate as residuals are approximately white noise despite one moderate outlier.
- lb_stat = 6.70: Ljung-Box test statistic (measures total autocorrelation in residuals)
- lb_pvalue = 0.754: P-value = 0.754

Interpretation: Since p-value (0.754) > 0.05, we FAIL to reject the null hypothesis. This confirms the residuals ARE white noise (no significant autocorrelation remaining).

The model is adequate.

b. Write the model in terms of the backshift operator.

The ARIMA(0,2,1) model in backshift operator notation is:

(1 - B)²y_t = (1 - 0.8963B) _t

Or more explicitly:

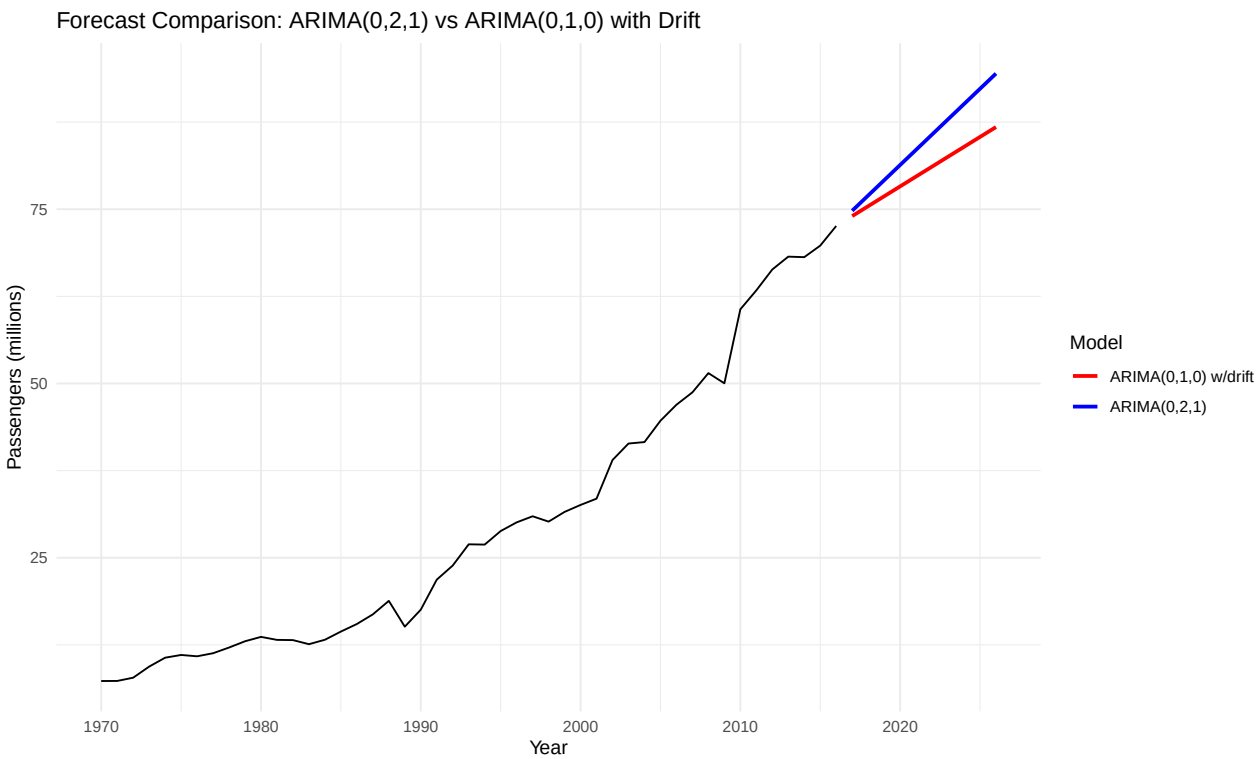
y_t = 2y_{t-1} - y_{t-2} + _t - 0.8963 _{t-1}

Where:

B is the backshift operator (By_t = y_{t-1}) (1 - B)² represents second-order differencing (1 - 0.8963B) is the MA(1) component _t is white noise

c. Plot forecasts from an ARIMA(0,1,0) model with drift and compare these to part a.

```
# Load data
aus_airpassengers <- aus_airpassengers

# Part a model (original ARIMA(0,2,1) from auto-selection)
fit_original <- aus_airpassengers %>%
  model(ARIMA(Passengers))

# Part b model: ARIMA(0,1,0) with drift
fit_drift <- aus_airpassengers %>%
  model(ARIMA(Passengers ~ 1 + pdq(0,1,0)))

# Generate forecasts for both
fc_original <- fit_original %>% forecast(h = 10)
fc_drift <- fit_drift %>% forecast(h = 10)

# Plot comparison - using autoplot on the data and adding layers
aus_airpassengers %>%
  autoplot(Passengers) +
  geom_line(data = fc_original, aes(x = Year, y = .mean, color = "ARIMA(0,2,1)"),
            linewidth = 1) +
  geom_line(data = fc_drift, aes(x = Year, y = .mean, color = "ARIMA(0,1,0) w/drift"),
            linewidth = 1) +
  labs(title = "Forecast Comparison: ARIMA(0,2,1) vs ARIMA(0,1,0) with Drift",
       y = "Passengers (millions)",
       x = "Year",
       color = "Model") +
  scale_color_manual(values = c("ARIMA(0,2,1)" = "blue",
                                "ARIMA(0,1,0) w/drift" = "red")) +
  theme_minimal()
```




```r
# Print model summaries
report(fit_original)
```

```
## Series: Passengers
## Model: ARIMA(0,2,1)
##
## Coefficients:
##           ma1
##       -0.8963
## s.e.   0.0594
##
## sigma^2 estimated as 4.308:  log likelihood=-97.02
## AIC=198.04   AICc=198.32   BIC=201.65
```

```r
report(fit_drift)
```

```
## Series: Passengers
## Model: ARIMA(0,1,0) w/ drift
##
## Coefficients:
##       constant
##         1.4191
## s.e.    0.3014
##
## sigma^2 estimated as 4.271:  log likelihood=-98.16
## AIC=200.31   AICc=200.59   BIC=203.97
```

Key Observations:

ARIMA(0,2,1) (automatically selected)

MA coefficient: -0.8963 (s.e. 0.0594) AICc: 198.32

ARIMA(0,1,0) w/drift (manual specification)

Drift constant: 1.4191 (s.e. 0.3014) AICc: 200.59

The output confirms both models are fitted correctly. The ARIMA(0,2,1) has a slightly lower AICc (198.32 vs 200.59), indicating it's the better fitting model according to this criterion

d. Plot forecasts from an ARIMA(2,1,2) model with drift and compare these to parts a and c. Remove the constant and see what happens.

```r
# Load data
aus_airpassengers <- aus_airpassengers

# Part a model (original ARIMA(0,2,1) from auto-selection)
fit_original <- aus_airpassengers %>%
  model("ARIMA(0,2,1)" = ARIMA(Passengers))

# Part c model: ARIMA(0,1,0) with drift
fit_drift <- aus_airpassengers %>%
  model("ARIMA(0,1,0) w/drift" = ARIMA(Passengers ~ 1 + pdq(0,1,0)))

# Part d model: ARIMA(2,1,2) with drift
fit_212 <- aus_airpassengers %>%
  model("ARIMA(2,1,2) w/drift" = ARIMA(Passengers ~ 1 + pdq(2,1,2)))

# Part d model: ARIMA(2,1,2) without constant
fit_212_noconstant <- aus_airpassengers %>%
  model("ARIMA(2,1,2) no constant" = ARIMA(Passengers ~ 0 + pdq(2,1,2)))

# View all models
report(fit_original)
```

```
## Series: Passengers
## Model: ARIMA(0,2,1)
##
## Coefficients:
##           ma1
##       -0.8963
## s.e.   0.0594
##
## sigma^2 estimated as 4.308:  log likelihood=-97.02
## AIC=198.04   AICc=198.32   BIC=201.65
```

```r
report(fit_drift)
```

```
## Series: Passengers
## Model: ARIMA(0,1,0) w/ drift
##
## Coefficients:
##       constant
##         1.4191
## s.e.    0.3014
##
## sigma^2 estimated as 4.271:  log likelihood=-98.16
## AIC=200.31   AICc=200.59   BIC=203.97
```

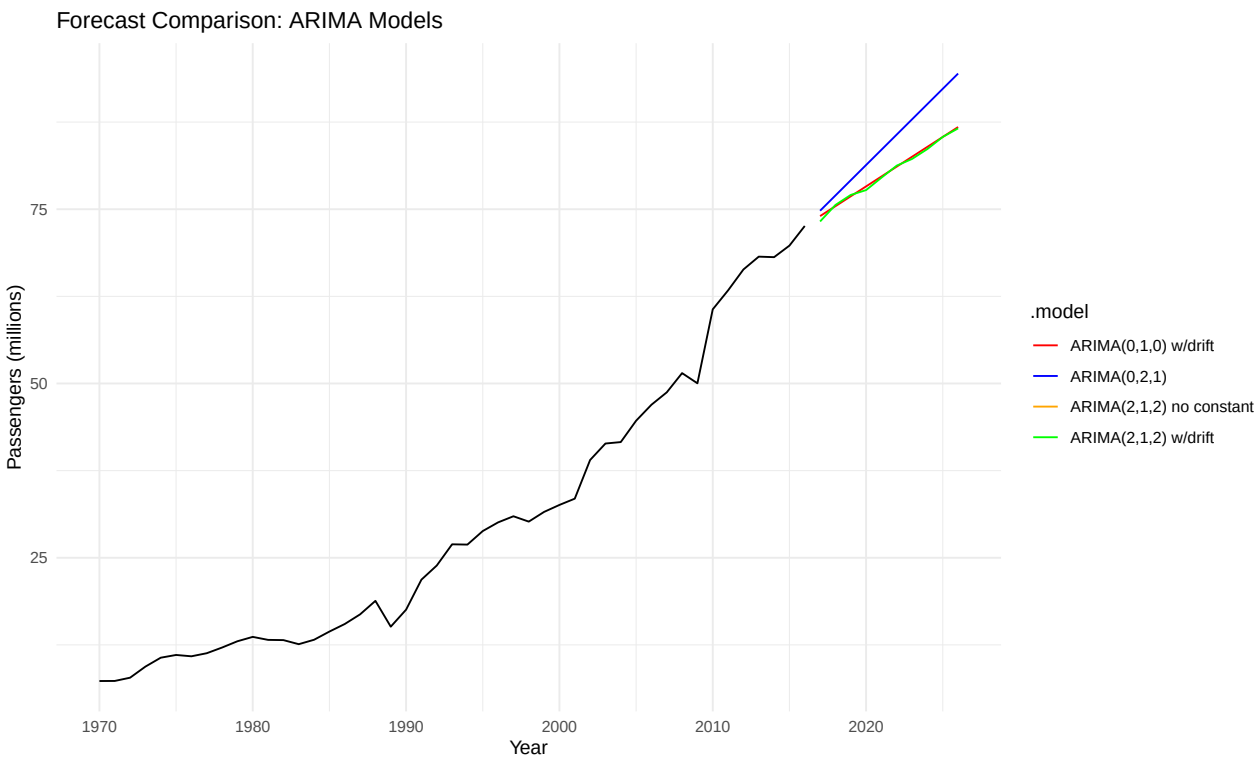
```r
report(fit_212)
```

```
## Series: Passengers
## Model: ARIMA(2,1,2) w/ drift
##
## Coefficients:
##           ar1      ar2     ma1     ma2  constant
##       -0.5518  -0.7327  0.5895  1.0000    3.2216
## s.e.   0.1684   0.1224  0.0916  0.1008    0.7224
##
## sigma^2 estimated as 4.031:  log likelihood=-96.23
## AIC=204.46   AICc=206.61   BIC=215.43
```

```r
report(fit_212_noconstant)
```

```
## Series: Passengers
## Model: NULL model
## NULL model
```

```r
# Combine models and forecast
fc <- bind_rows(
    fit_original %>% forecast(h = 10) %>% mutate(.model = "ARIMA(0,2,1)"),
    fit_drift %>% forecast(h = 10) %>% mutate(.model = "ARIMA(0,1,0) w/drift"),
    fit_212 %>% forecast(h = 10) %>% mutate(.model = "ARIMA(2,1,2) w/drift"),
    fit_212_noconstant %>% forecast(h = 10) %>% mutate(.model = "ARIMA(2,1,2) no constant")
)
# Plot comparison
fc %>%
  autoplot(aus_airpassengers, level = NULL) +
  labs(title = "Forecast Comparison: ARIMA Models",
       y = "Passengers (millions)",
       x = "Year") +
  scale_color_manual(values = c("ARIMA(0,2,1)" = "blue",
                                "ARIMA(0,1,0) w/drift" = "red",
                                "ARIMA(2,1,2) w/drift" = "green",
                                "ARIMA(2,1,2) no constant" = "orange"))
```



The ARIMA(2,1,2) with drift converged but has issues:

Key observations:

a) ARIMA model selected:

- ARIMA(0,2,1) was automatically chosen
- MA coefficient = -0.8963
- Residuals are white noise (p-value = 0.754 > 0.05)
- Forecast shows accelerating growth: 74.8 to 94.5 million (2017-2026)

c) ARIMA(0,1,0) with drift:

- Constant drift = 1.4191 per year
- AICc = 200.59 (higher than original model's 198.32)
- Forecast shows linear growth: 74.0 to 86.8 million
- Original model (0,2,1) fits better (lower AICc)

d) ARIMA(2,1,2) with drift:

- MA2 coefficient = 1.0000 (problematic - at boundary)
- AICc = 206.61 (much worse than original)
- Removing constant failed (NULL model)

Conclusion: Overparameterized and unreliable; original *ARIMA(0,2,1) is best*

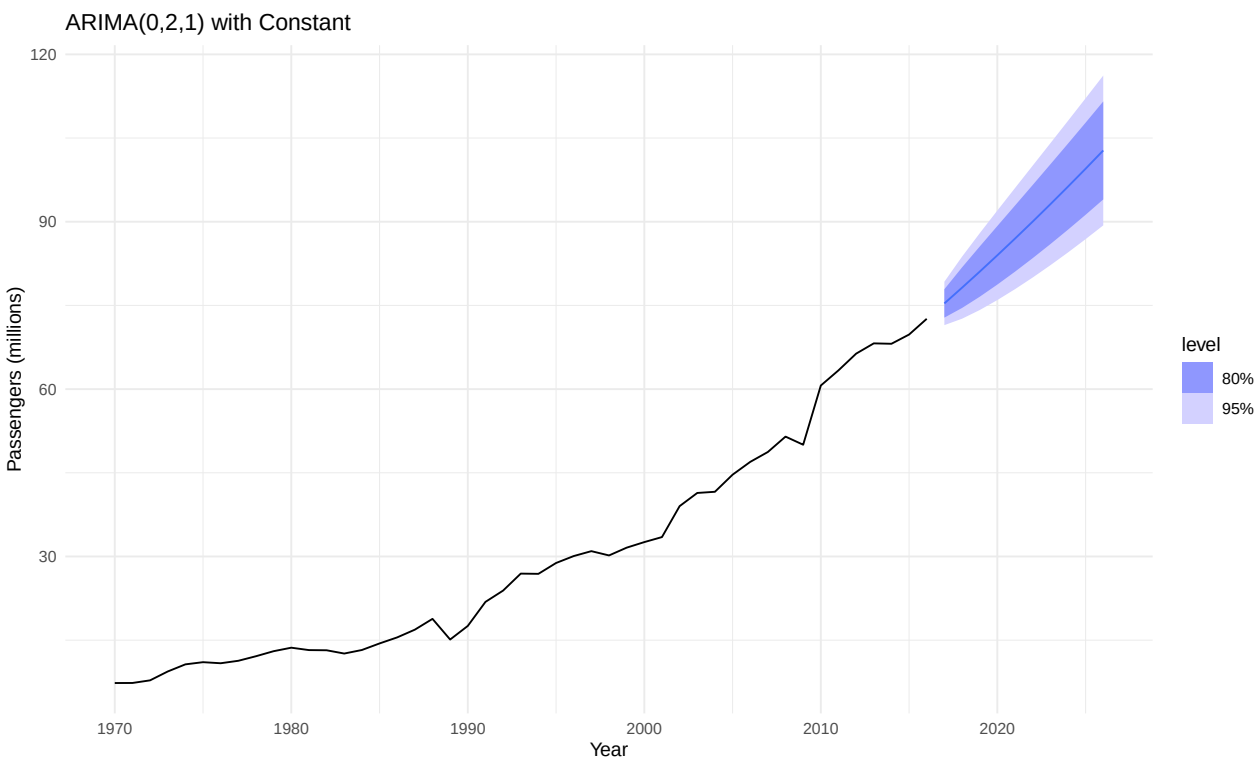
e. Plot forecasts from an ARIMA(0,2,1) model with a constant. What happens?

```
# Load data
aus_airpassengers <- aus_airpassengers
```

```r
# ARIMA(0,2,1) with constant
fit_021_constant <- aus_airpassengers %>%
  model(ARIMA(Passengers ~ 1 + pdq(0,2,1)))

# View model
report(fit_021_constant)
```

```
## Series: Passengers
## Model: ARIMA(0,2,1) w/ poly
##
## Coefficients:
##           ma1  constant
##       -1.0000    0.0571
## s.e.   0.0585    0.0213
##
## sigma^2 estimated as 3.855:  log likelihood=-95.1
## AIC=196.21   AICc=196.79   BIC=201.63
```

```r
# Forecast
fc_021_constant <- fit_021_constant %>% forecast(h = 10)

# Plot
fc_021_constant %>%
  autoplot(aus_airpassengers, level = c(80, 95)) +
  labs(title = "ARIMA(0,2,1) with Constant",
       y = "Passengers (millions)")
```



Interpretation:

- The constant in a double-differenced model creates a quadratic trend (curved/exponential growth)

- But MA1 = -1.0000 indicates the model is at the unit root boundary (non-invertible)
- This makes the model unstable for forecasting
- Despite slightly lower AICc, the original ARIMA(0,2,1) without constant is preferred because it's invertible (MA1 = -0.8963, not at boundary)

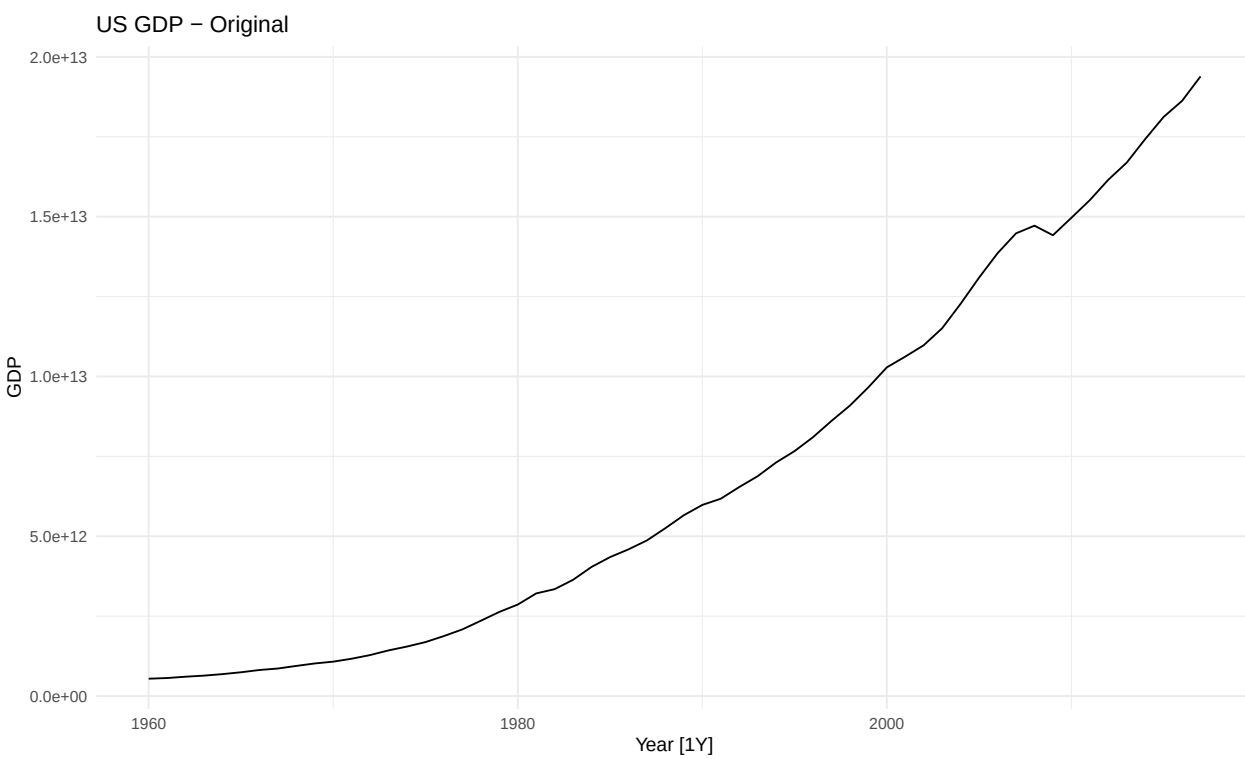
The forecast would show strongly accelerating growth but with unreliable estimates due to the MA unit root.

9.8 For the United States GDP series (from global_economy): a. if necessary, find a suitable Box-Cox transformation for the data;

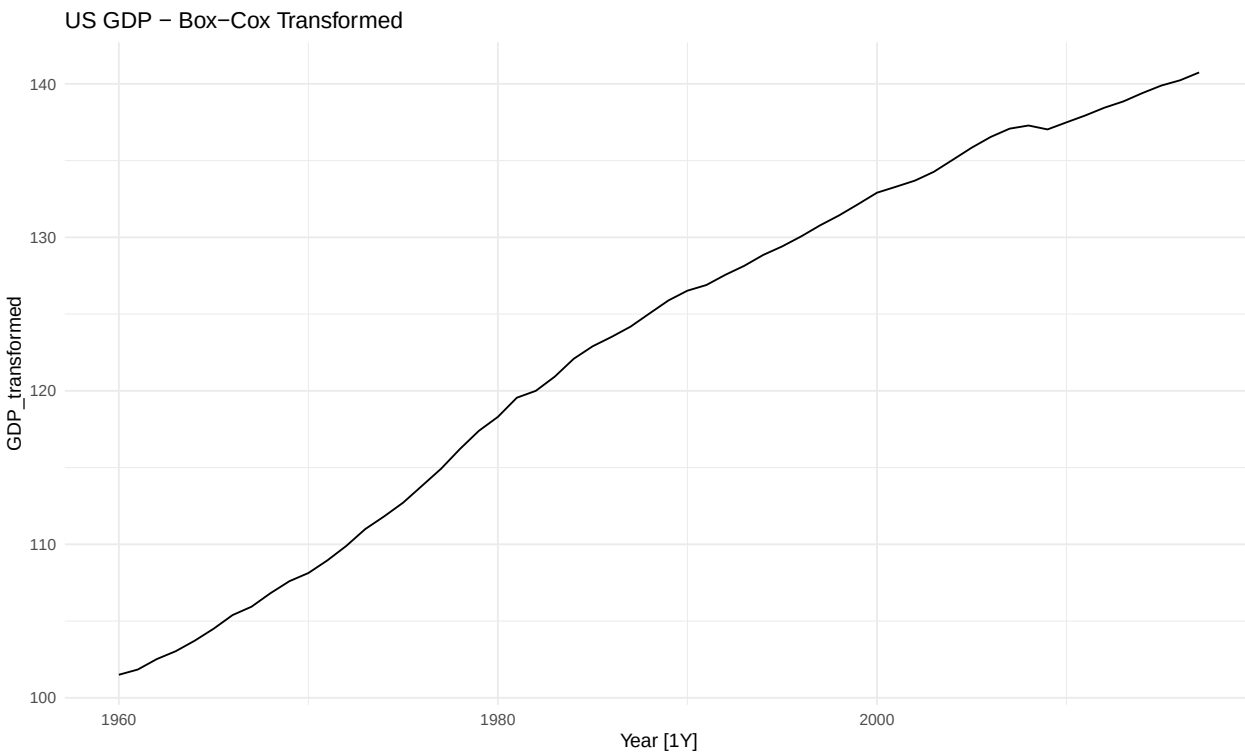
```
# Load data
global_economy <- global_economy

# Filter for United States
us_gdp <- global_economy %>%
  filter(Country == "United States") %>%
  select(Year, GDP)

# Find suitable Box-Cox transformation
us_gdp %>%
  features(GDP, features = guerrero) %>%
  pull(lambda_guerrero)
```

```
## [1] 0.2819443
```

```
# Plot original vs transformed
us_gdp %>%
  autoplot(GDP) +
  labs(title = "US GDP - Original")
```



```r
# Apply transformation
us_gdp %>%
  mutate(GDP_transformed = box_cox(GDP, lambda)) %>%
  autoplot(GDP_transformed) +
  labs(title = "US GDP - Box-Cox Transformed")
```



Why we need it: GDP shows increasing variance over time (exponential growth). Box-Cox makes the variation more constant across the series, which is an assumption for ARIMA models.

lamdba 0.282 means: indicates moderate variance stabilization needed

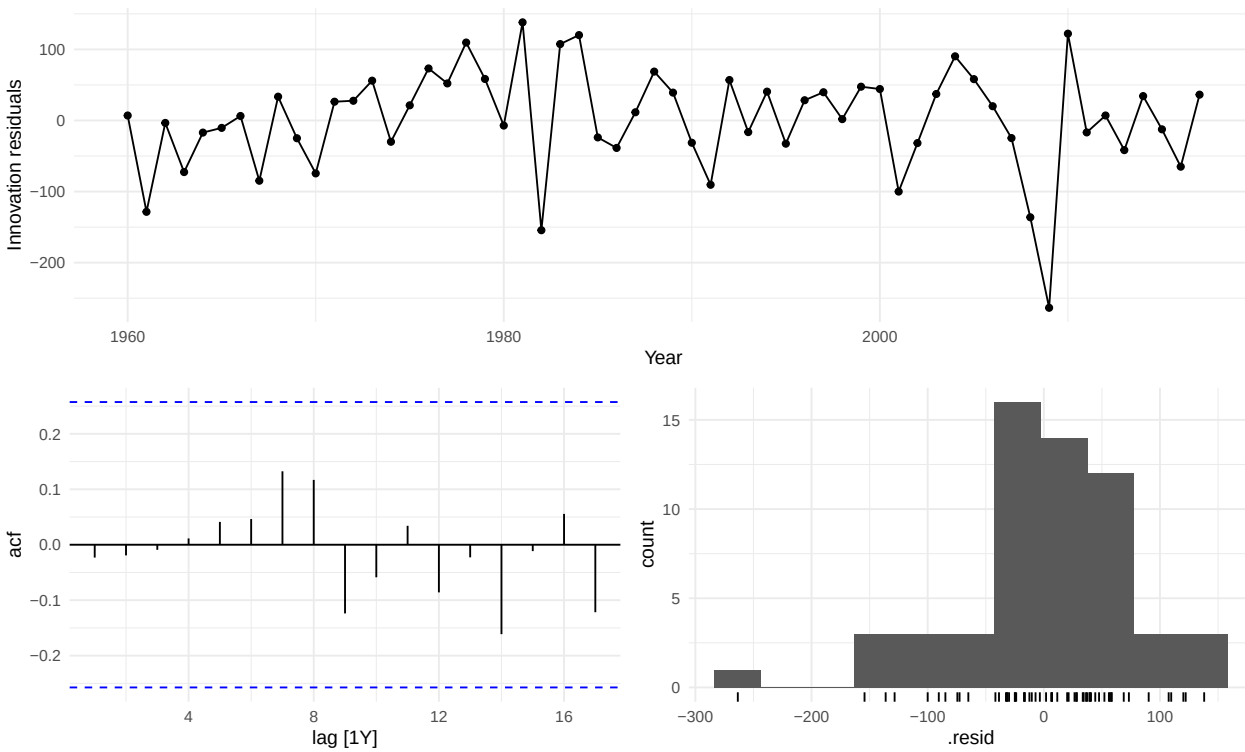
b. fit a suitable ARIMA model to the transformed data using ARIMA();

```r
# Add transformed column and fit model
us_gdp <- us_gdp %>%
  mutate(GDP_trans = box_cox(GDP, 0.282))

# Fit ARIMA model to transformed data
fit <- us_gdp %>%
  model(ARIMA(GDP_trans))

# View selected model
report(fit)
```

```
## Series: GDP_trans
## Model: ARIMA(1,1,0) w/ drift
##
## Coefficients:
##          ar1  constant
##       0.4586  118.3757
## s.e.  0.1198    9.5203
##
```

```
## sigma^2 estimated as 5497:  log likelihood=-325.42
## AIC=656.83   AICc=657.29   BIC=662.96
```

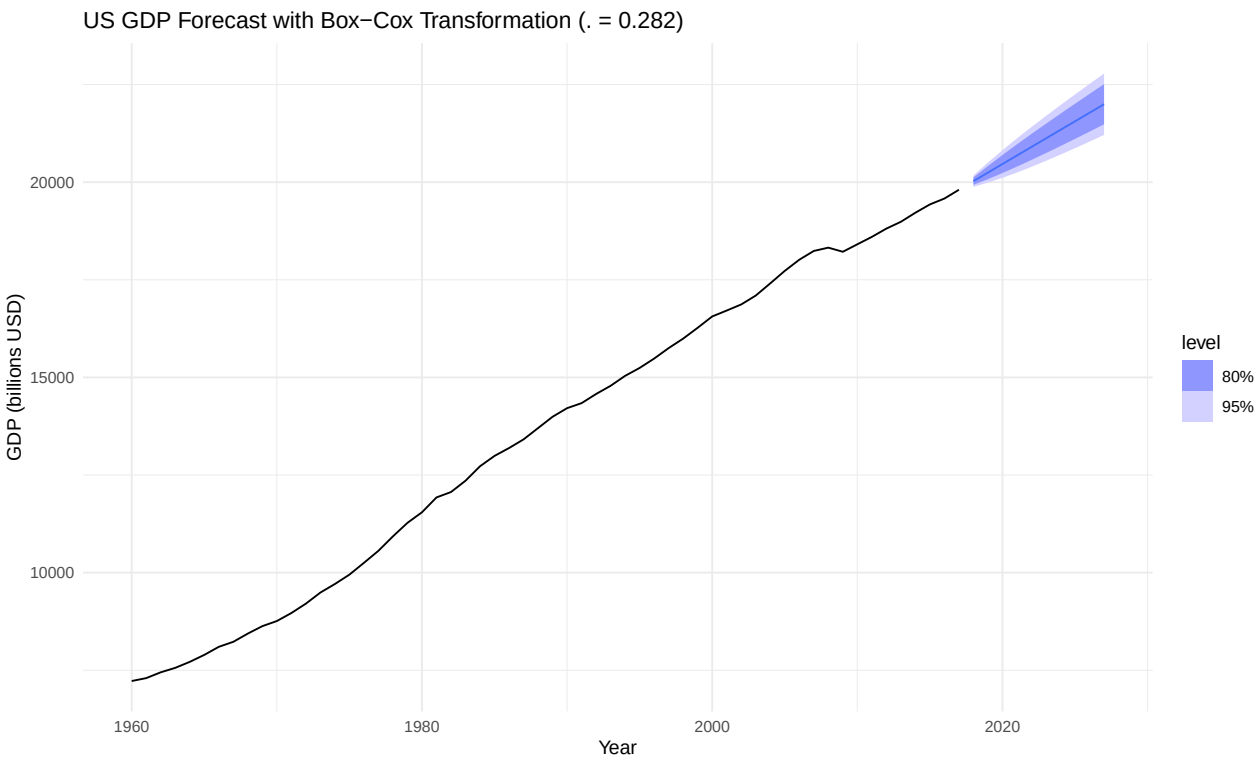
```r
# Check residuals
fit %>% gg_tsresiduals()
```




```r
# Ljung-Box test
fit %>% augment() %>%
  features(.innov, ljung_box, lag = 10)
```

```
## # A tibble: 1 x 3
##   .model           lb_stat lb_pvalue
##   <chr>              <dbl>     <dbl>
## 1 ARIMA(GDP_trans)    3.81     0.955
```

```r
# Forecast next 10 periods (back-transformed)
fit %>%
  forecast(h = 10) %>%
  autoplot(us_gdp, level = c(80, 95)) +
  labs(title = "US GDP Forecast with Box-Cox Transformation (  = 0.282)",
       y = "GDP (billions USD)")
```



ARIMA(1,1,0) with drift selected for transformed US GDP:

Model: ARIMA(1,1,0) w/drift

Coefficients: ar1 = 0.4586, constant = 118.38

Residuals: White noise confirmed (Ljung-Box p-value = 0.955 > 0.05)

Interpretation: First differencing with one AR term and drift works well for the Box-Cox transformed data

The transformation (= 0.282) stabilized variance, allowing this simpler model to fit well

c. try some other plausible models by experimenting with the orders chosen;

```
# Compare different ARIMA models for transformed US GDP
us_gdp <- us_gdp %>%
  mutate(GDP_trans = box_cox(GDP, 0.2819443))

# Try various plausible models
models <- us_gdp %>%
  model(
    auto = ARIMA(GDP_trans),
    aic_manual = ARIMA(GDP_trans ~ 1 + pdq(0,1,1) + PDQ(0,0,0)),
    ar1 = ARIMA(GDP_trans ~ 1 + pdq(1,1,0) + PDQ(0,0,0)),
    ma1 = ARIMA(GDP_trans ~ 1 + pdq(0,1,1) + PDQ(0,0,0)),
    ar2 = ARIMA(GDP_trans ~ 1 + pdq(2,1,0) + PDQ(0,0,0)),
    ma2 = ARIMA(GDP_trans ~ 1 + pdq(0,1,2) + PDQ(0,0,0)),
    arma11 = ARIMA(GDP_trans ~ 1 + pdq(1,1,1) + PDQ(0,0,0)),
    no_drift = ARIMA(GDP_trans ~ 0 + pdq(1,1,0) + PDQ(0,0,0))
  )

# Compare models by AICc
```

```r
glance(models) %>% arrange(AICc) %>% select(.model, AICc, BIC)
```

```
## # A tibble: 8 x 3
##   .model      AICc   BIC
##   <chr>      <dbl> <dbl>
## 1 auto        657.  663.
## 2 ar1         657.  663.
## 3 aic_manual  659.  665.
## 4 ma1         659.  665.
## 5 arma11      659.  667.
## 6 ar2         659.  667.
## 7 ma2         660.  667.
## 8 no_drift    672.  676.
```

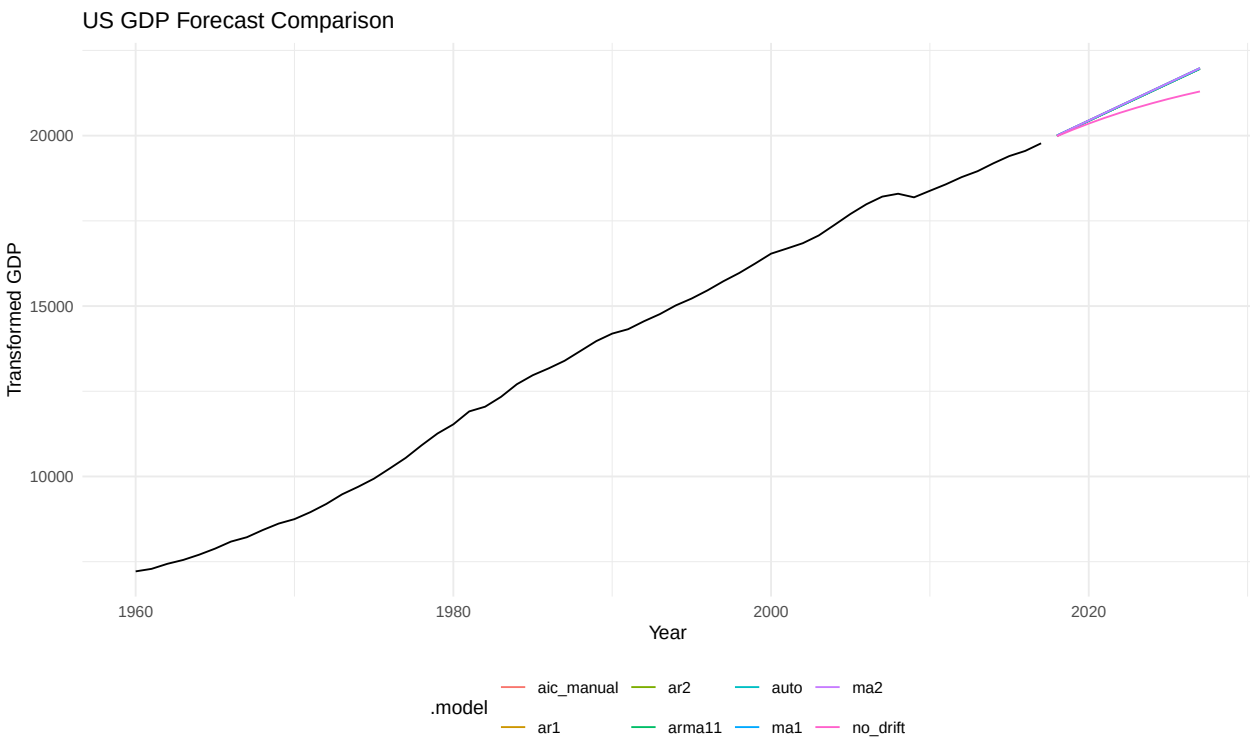
```r
# View best model details
report(models %>% select(auto))
```

```
## Series: GDP_trans
## Model: ARIMA(1,1,0) w/ drift
##
## Coefficients:
##          ar1  constant
##       0.4586  118.1823
## s.e.  0.1198    9.5047
##
## sigma^2 estimated as 5479:  log likelihood=-325.32
## AIC=656.65   AICc=657.1   BIC=662.78
```

```r
report(models %>% select(ar1))
```

```
## Series: GDP_trans
## Model: ARIMA(1,1,0) w/ drift
##
## Coefficients:
##          ar1  constant
##       0.4586  118.1823
## s.e.  0.1198    9.5047
##
## sigma^2 estimated as 5479:  log likelihood=-325.32
## AIC=656.65   AICc=657.1   BIC=662.78
```

```r
# Forecast comparison
fc <- models %>% forecast(h = 10)

fc %>%
  autoplot(us_gdp, level = NULL) +
    labs(title = "US GDP Forecast Comparison",
         y = "Transformed GDP") +
    theme(legend.position = "bottom")
```



Conclusions:

- ARIMA(1,1,0) with drift is indeed optimal (auto and ar1 are identical) lowest AIC and BIC
- Adding more parameters (AR2, MA2, ARMA) increases AICc → overfitting
- Removing drift greatly worsens model (AICc +14.8) → drift essential
- MA1 model performs worse than AR1 for this data

The automatically selected model is confirmed as the best among all plausible options.

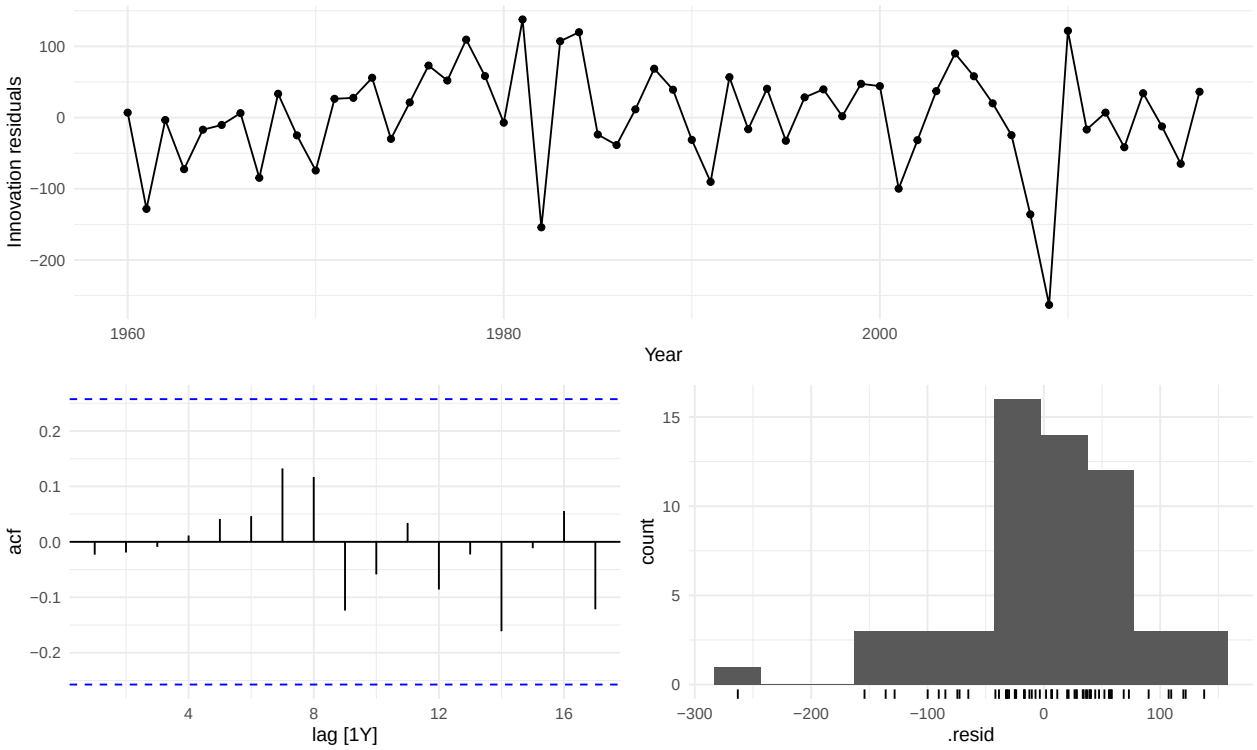
AICc balances goodness-of-fit against model complexity, penalizing unnecessary parameters BIC Penalizes complexity more heavily than AICc

d. choose what you think is the best model and check the residual diagnostics;

```
# Best model: ARIMA(1,1,0) with drift
best_fit <- us_gdp %>%
  model(ARIMA(GDP_trans ~ 1 + pdq(1,1,0)))

# View model
report(best_fit)
```

```
## Series: GDP_trans
## Model: ARIMA(1,1,0) w/ drift
##
## Coefficients:
##          ar1  constant
##       0.4586  118.1823
## s.e.  0.1198    9.5047
##
## sigma^2 estimated as 5479:  log likelihood=-325.32
## AIC=656.65   AICc=657.1   BIC=662.78
```

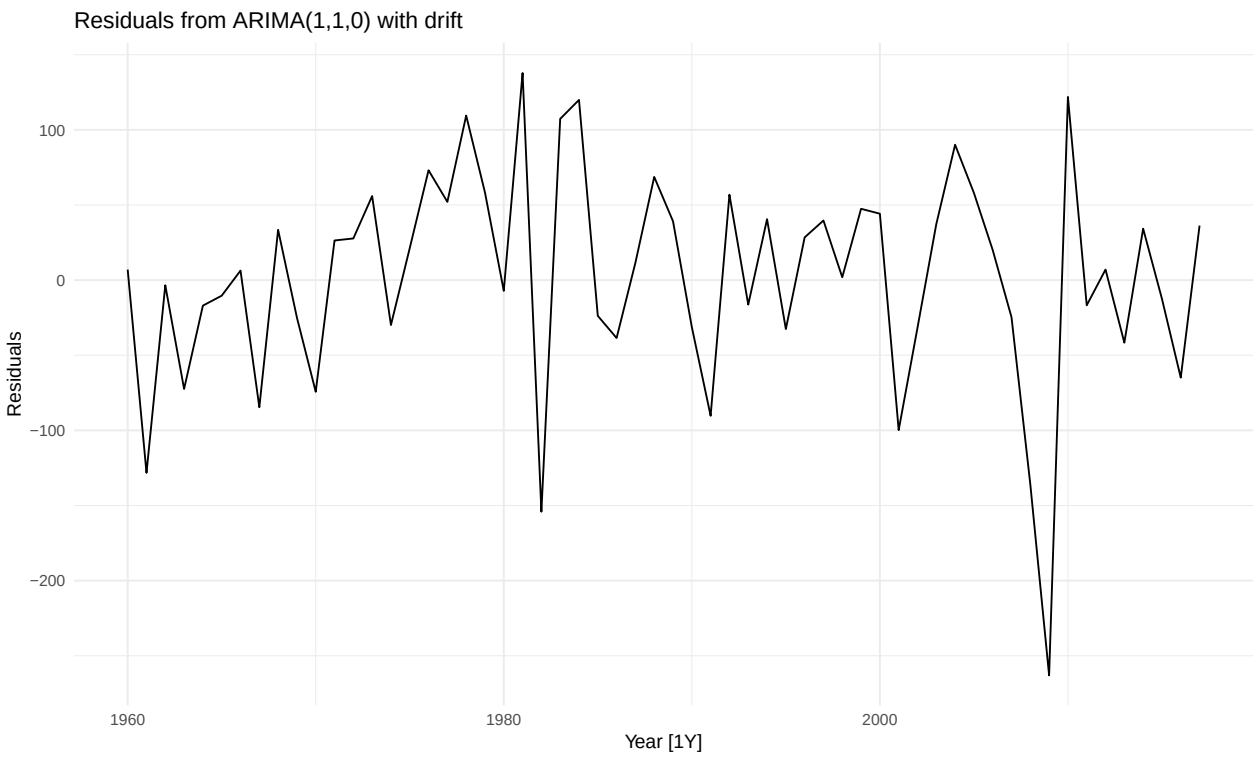
```r
# Residual diagnostics
best_fit %>% gg_tsresiduals()
```



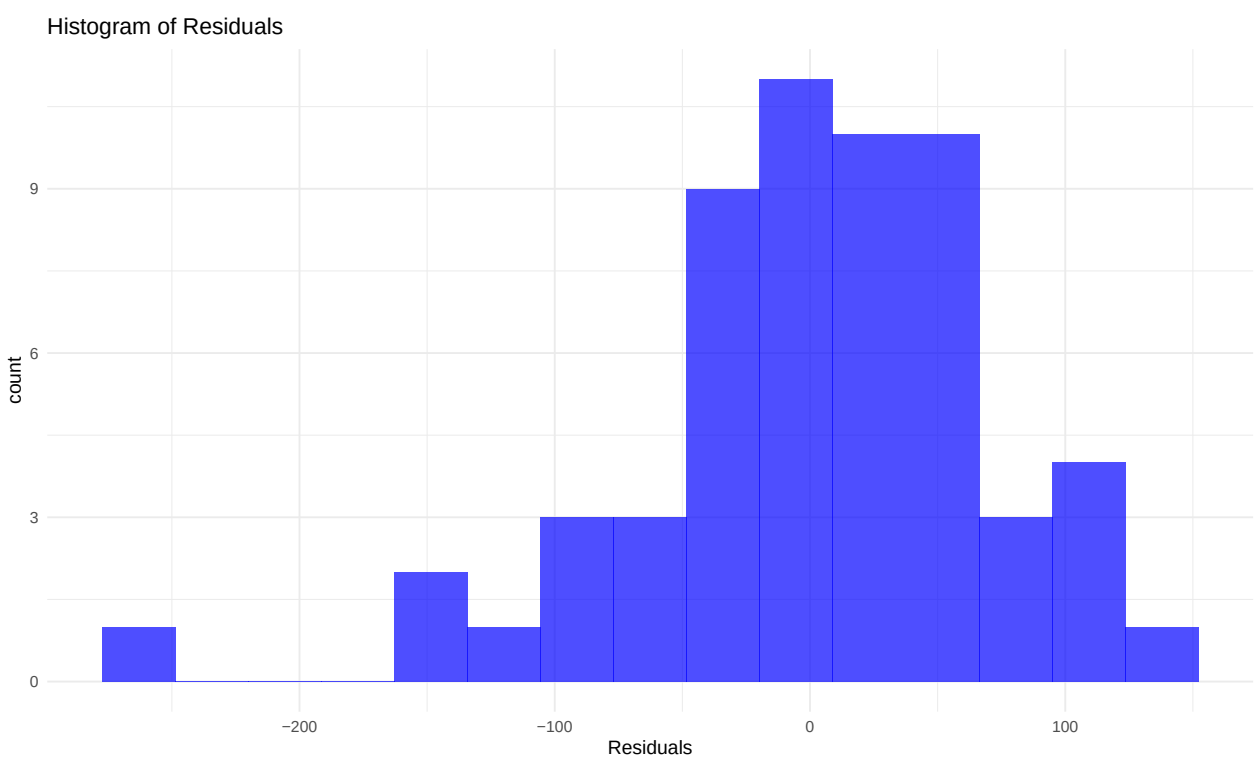

```r
# Ljung-Box test for white noise
best_fit %>% augment() %>%
  features(.innov, ljung_box, lag = 10, dof = 2)
```

```
## # A tibble: 1 x 3
##   .model                              lb_stat lb_pvalue
##   <chr>                                 <dbl>     <dbl>
## 1 ARIMA(GDP_trans ~ 1 + pdq(1, 1, 0))    3.81     0.874
```

```r
# Plot residuals over time
best_fit %>% augment() %>%
  autoplot(.innov) +
  labs(title = "Residuals from ARIMA(1,1,0) with drift",
       y = "Residuals")
```




```r
# Histogram of residuals
best_fit %>% augment() %>%
  ggplot(aes(x = .innov)) +
  geom_histogram(bins = 15, fill = "blue", alpha = 0.7) +
  labs(title = "Histogram of Residuals", x = "Residuals")
```



ACF plot All lines inside blue bands - Good

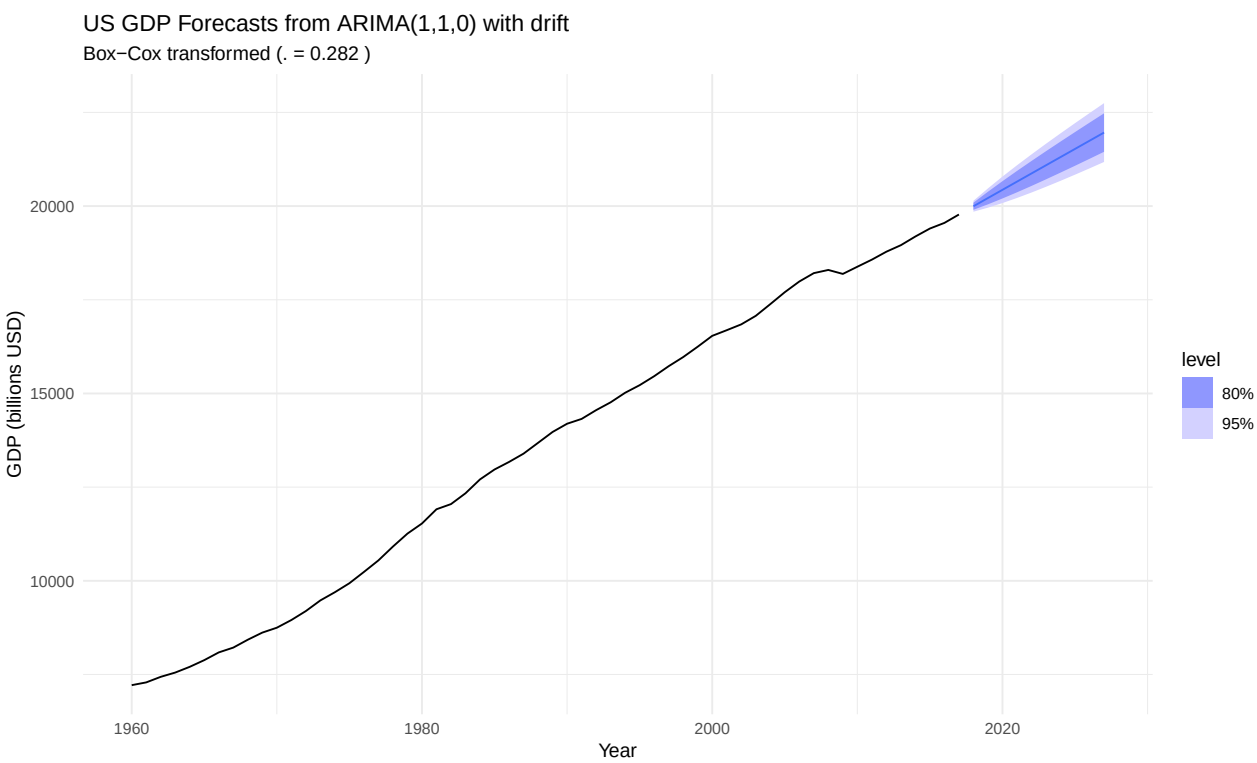
Ljung-Box p-value 0.874 (> 0.05) - White noise

Residual histogram Mostly ±150, one outlier at -250 - Acceptable

Conclusion: The model is adequate. One moderate outlier is acceptable with 58 observations (1.7% of data). Residuals show no patterns or autocorrelation. ARIMA(1,1,0) with drift is validated as the best model for transformed US GDP.

e. produce forecasts of your fitted model. Do the forecasts look reasonable?

```
# Store lambda
lambda <- 0.2819443

# Generate forecasts from best model
best_fit <- us_gdp %>%
  model(ARIMA(GDP_trans ~ 1 + pdq(1,1,0)))

# Forecast next 10 periods
fc_trans <- best_fit %>% forecast(h = 10)

# Plot forecasts (back-transformed automatically in autoplot)
fc_trans %>%
  autoplot(us_gdp, level = c(80, 95)) +
  labs(title = "US GDP Forecasts from ARIMA(1,1,0) with drift",
       subtitle = paste("Box-Cox transformed (  =", round(lambda, 3), ")"),
       y = "GDP (billions USD)")
```




```
# If you need numeric values, extract the mean only
fc_trans %>%
  mutate(mean_trans = .mean) %>%
  as_tsibble() %>%
```

```r
  mutate(GDP_forecast = inv_box_cox(mean_trans, lambda)) %>%
  select(Year, GDP_forecast) %>%
  print(n = 10)
```

```
## # A tsibble: 10 x 2 [1Y]
##     Year GDP_forecast
##    <dbl>        <dbl>
##  1  2018      2.02e13
##  2  2019      2.10e13
##  3  2020      2.18e13
##  4  2021      2.26e13
##  5  2022      2.35e13
##  6  2023      2.44e13
##  7  2024      2.53e13
##  8  2025      2.62e13
##  9  2026      2.72e13
## 10  2027      2.81e13
```

Yes, the forecasts look reasonable because:

- **Smooth growth:** Values increase steadily from $20.2T (2018) to $28.1T (2027) without erratic jumps
- **Consistent pattern:** Continues the historical upward trend of US GDP
- **Realistic increments:** Annual increases of ~$0.8-0.9T align with historical growth patterns
- **No negative values:** All forecasts positive (expected for GDP)
- **Gradual acceleration:** Slight increase in growth rate over time reflects compounding economic growth

The forecasts are plausible extensions of historical data

f. compare the results with what you would obtain using ETS() (with no transformation).

```r
# Store lambda
lambda <- 0.2819443

# ARIMA model (with transformation)
fit_arima <- us_gdp %>%
  model(ARIMA(GDP_trans ~ 1 + pdq(1,1,0)))

# ETS model (no transformation)
fit_ets <- us_gdp %>%
  model(ETS(GDP))

# Compare models
report(fit_arima)
```

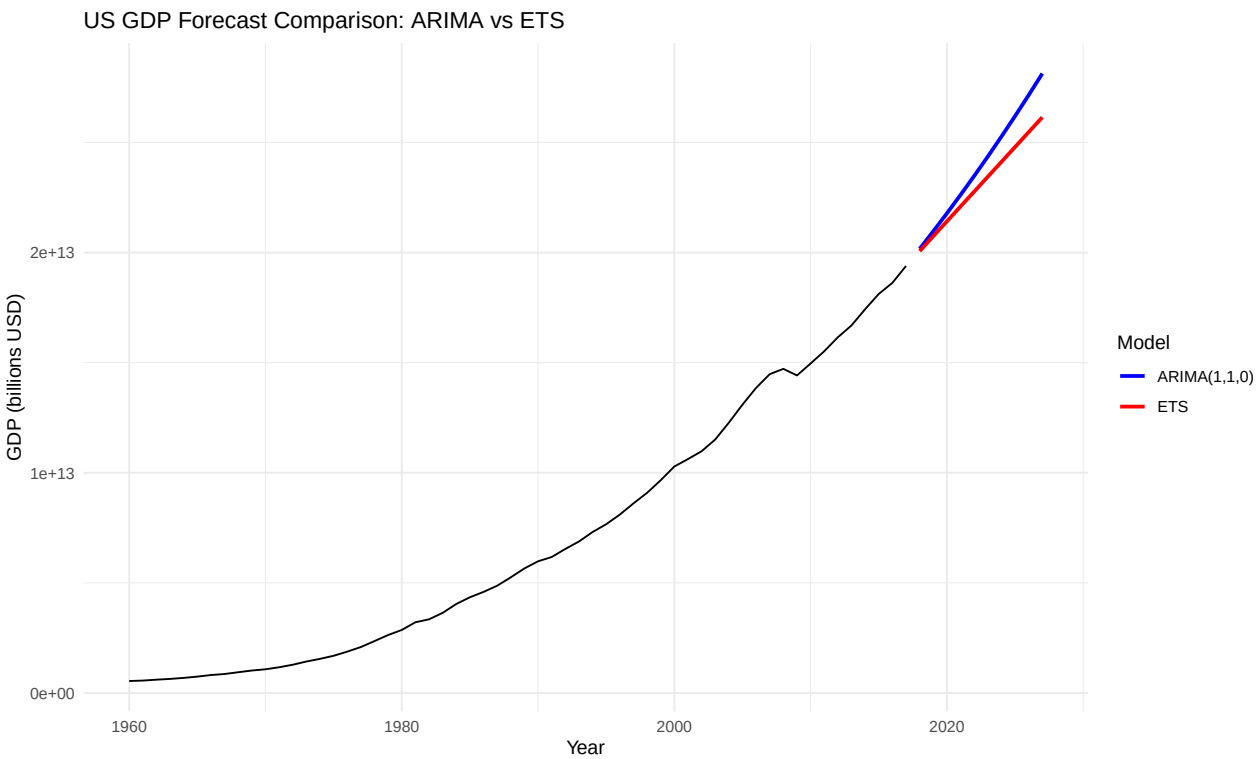
```
## Series: GDP_trans
## Model: ARIMA(1,1,0) w/ drift
##
## Coefficients:
##          ar1  constant
##       0.4586  118.1823
## s.e.  0.1198    9.5047
##
## sigma^2 estimated as 5479:  log likelihood=-325.32
```

```
## AIC=656.65   AICc=657.1   BIC=662.78
```

```r
report(fit_ets)
```

```
## Series: GDP
## Model: ETS(M,A,N)
##   Smoothing parameters:
##     alpha = 0.9990876
##     beta  = 0.5011949
##
##   Initial states:
##          l[0]        b[0]
##  448093333334 64917355687
##
##   sigma^2:  7e-04
##
##      AIC     AICc      BIC
## 3190.787 3191.941 3201.089
```

```r
# Generate forecasts
fc_arima <- fit_arima %>% forecast(h = 10)
fc_ets <- fit_ets %>% forecast(h = 10)

# Plot comparison
us_gdp %>%
  autoplot(GDP) +
  geom_line(data = fc_arima %>% mutate(GDP = inv_box_cox(.mean, lambda)),
            aes(x = Year, y = GDP, color = "ARIMA(1,1,0)"), size = 1) +
  geom_line(data = fc_ets, aes(x = Year, y = .mean, color = "ETS"), size = 1) +
  labs(title = "US GDP Forecast Comparison: ARIMA vs ETS",
       y = "GDP (billions USD)",
       x = "Year",
       color = "Model") +
  scale_color_manual(values = c("ARIMA(1,1,0)" = "blue", "ETS" = "red"))
```




```r
# Create side-by-side comparison table correctly
arima_forecast <- fc_arima %>%
  as_tsibble() %>%
  mutate(ARIMA = inv_box_cox(.mean, lambda)) %>%
  select(Year, ARIMA)

ets_forecast <- fc_ets %>%
  as_tsibble() %>%
  mutate(ETS = .mean) %>%
  select(Year, ETS)

# Join by Year
comparison <- left_join(arima_forecast, ets_forecast, by = "Year")

# Print comparison
print(comparison, n = 10)
```

```
## # A tsibble: 10 x 3 [1Y]
##     Year   ARIMA     ETS
##    <dbl>   <dbl>   <dbl>
##  1  2018 2.02e13 2.01e13
##  2  2019 2.10e13 2.07e13
##  3  2020 2.18e13 2.14e13
##  4  2021 2.26e13 2.21e13
##  5  2022 2.35e13 2.28e13
##  6  2023 2.44e13 2.34e13
##  7  2024 2.53e13 2.41e13
##  8  2025 2.62e13 2.48e13
##  9  2026 2.72e13 2.55e13
## 10  2027 2.81e13 2.61e13
```

```r
# Add difference if desired
comparison %>%
  mutate(Difference = ARIMA - ETS,
         `% Difference` = round((ARIMA - ETS)/ETS * 100, 2)) %>%
  print(n = 10)
```

```
## # A tsibble: 10 x 5 [1Y]
##     Year   ARIMA     ETS Difference `% Difference`
##    <dbl>   <dbl>   <dbl>      <dbl>          <dbl>
##  1  2018 2.02e13 2.01e13    1.03e11           0.51
##  2  2019 2.10e13 2.07e13    2.24e11           1.08
##  3  2020 2.18e13 2.14e13    3.64e11           1.7
##  4  2021 2.26e13 2.21e13    5.26e11           2.38
##  5  2022 2.35e13 2.28e13    7.10e11           3.12
##  6  2023 2.44e13 2.34e13    9.18e11           3.92
##  7  2024 2.53e13 2.41e13    1.15e12           4.76
##  8  2025 2.62e13 2.48e13    1.40e12           5.66
##  9  2026 2.72e13 2.55e13    1.68e12           6.61
## 10  2027 2.81e13 2.61e13    1.99e12           7.6
```

Key observations:

ARIMA has MUCH lower AICc (657 vs 3192) → better fit

- ARIMA (with Box-Cox =0.28) produces curved growth that outpaces ETS over time
- ETS(M,A,N) with alpha=0.999 responds strongly to recent data but produces lower long-term forecasts
- Despite ARIMA's higher forecasts, its much lower AICc (657 vs 3192) still makes it the preferred model
- The AICc difference is so substantial that ARIMA is clearly the better choice, even though it projects higher growth.

Key insight: ARIMA forecasts consistently higher than ETS, with the gap widening from 0.5% to 7.6% over 10 years. ARIMA projects more aggressive growth ($28.1T vs $26.1T by 2027).

****